

INDEX

TOPICS	Page No's
➤ Certificates	
➤ Acknowledgement	
➤ Abstract	
➤ Figures/Tables	
CHAPTER-1: INTRODUCTION	1-2
CHAPTER-2: LITERATURE SURVEY	3-6
CHAPTER-3: SYSTEM ANALYSIS	
3.1 Existing System	7
3.2 Proposed System	8-9
CHAPTER-4: SYSTEM REQUIREMENTS	
4.1 Functional Requirement	10
4.2 Non-Functional Requirements	10
CHAPTER-5: SYSTEM STUDY	
5.1 Feasibility Study	11
5.2 Feasibility Analysis	11-12
CHAPTER-6: SYSTEM DESIGN	
6.1 SYSTEM ARCHITECTURE	13
6.2 UML DIAGRAMS	13-21
6.2.1 Use Case Diagram	
6.2.2 Class Diagram	

- 6.2.3 Sequence Diagram
- 6.2.4 Collaboration Diagram
- 6.2.5 Activity Diagram
- 6.2.6 Component Diagram
- 6.2.7 Deployment Diagram
- 6.2.8 Er Diagram
- 6.2.9 Data Dictionary

CHAPTER-7: INPUT AND OUTPUT DESIGN

- 7.1 Input Design 22
- 7.2 Output Design 23

CHAPTER-8: IMPLEMENTATION

- 8.1 Modules 24
 - 8.1.1 Module Description 24

CHAPTER-9: SOFTWARE ENVIRONMENT

- 9.1 Python 25-68
- 9.2 Source Code 68-75

CHAPTER-10: RESULTS/DISCUSSIONS

- 10.1 System Testing 76-79
 - 10.1.1 Test Cases 79-82
- 10.2 Screen shots 82-86

CHAPTER-11: CONCLUSION

11.1 Conclusion 87

11.2 Future Scope 87-88

CHAPTER-12: REFERENCES 89-91

LIST OF FIGURES

S.NO	TABLES/FIGURES	PAGE NO'S
1	System Architecture	13
2	UML Diagrams	14-21
	2.1 Use Case Diagram	14
	2.2 Class Diagram	15
	2.3 Sequence Diagram	16
	2.4 Collaboration Diagram	16
	2.5 Activity Diagram	17
	2.6 Component Diagram	18
	2.7 Deployment Diagram	18
	2.8 ER Diagram	19
	2.9 Data Dictionary/Data Set	19-21
3	Compiling and interpretation of Python Source code	25-68
4	Screenshots	82-86

DEEP SIDE A DEEP LEARNING FRAMEWORK FOR DRUG SIDE EFFECT PREDICTION

ABSTRACT

Drug failures due to unforeseen adverse effects at clinical trials pose health risks for the participants and lead to substantial financial losses. Side effect prediction algorithms have the potential to guide the drug design process. LINCS L1000 dataset provides a vast resource of cell line gene expression data perturbed by different drugs and creates a knowledge base for context specific features. The state-of-the-art approach that aims at using context specific information relies on only the high quality experiments in LINCS L1000 and discards a large portion of the experiments. In this study, our goal is to boost the prediction performance by utilizing this data to its full extent. We experiment with 5 deep learning architectures. We find that a multi-modal architecture produces the best predictive performance among multi-layer perceptron-based architectures when drug chemical structure (CS), and the full set of drug perturbed gene expression profiles (GEX) are used as modalities. Overall, we observe that the CS is more informative than the GEX. A convolutional neural network-based model that uses only SMILES string representation of the drugs achieves the best results and provides 13:0% macro-AUC and 3:1% micro-AUC improvements over the state-of-the-art. We also show that the model is able to predict side effect-drug pairs that are reported in the literature but was missing in the ground truth side effect dataset.

CHAPTER-1

INTRODUCTION

Computational methods hold great promise for mitigating the health and financial risks of drug development by predicting possible side effects before entering into the clinical trials. Several learning based methods have been proposed for predicting the side effects of drugs based on various features such as: chemical structures of drugs [25, 1, 23, 8, 19, 34, 17, 9, 2, 5], drug-protein interactions [35, 33, 8, 19, 34, 17, 37, 2, 15, 36], protein-protein interactions (PPI) [8, 9], activity in metabolic networks [38, 26], pathways, phenotype information and gene annotations [8]. In parallel to the above mentioned approaches, recently, deep learning models have been employed to predict side effects: (i) [31] uses biological, chemical and semantic information on drugs in addition to clinical notes and case reports and (ii) [4] uses various chemical fingerprints extracted using deep architectures to compare the side effect prediction performance.

While these methods have proven useful for predicting adverse drug reactions (ADRs – used interchangeably with drug side effects), the features they use are solely based on external knowledge about the drugs (i.e., drug-protein interactions, etc.) and are not cell or condition (i.e., dosage) specific. To address this issue, Wang et al. (2016) utilize the data from the LINCS L1000 project [32]. This project profiles gene expression changes in numerous human cell lines after treating them with a large number of drugs and small-molecule compounds. By using the gene expression profiles of the treated cells, [32] provides the first comprehensive, unbiased, and cost-effective prediction of ADRs. The paper formulates the problem as a multi-label classification task. Their results suggest that the gene expression profiles provide context-dependent information for the side-effect prediction task. While the LINCS dataset contains a total of 473,647 experiments for 20,338 compounds, their method utilizes only the highest quality experiment for each drug to minimize noise. This means that most of the expression data are left unused, suggesting a potential room for improvement in the prediction performance. Moreover, their framework performs feature engineering by transforming gene expression features to enrichment vectors of biological terms. In this work, we investigate whether the incorporation of gene expression data along with the drug structure data can be leveraged better in a deep learning framework without the need for feature engineering.

In this study, we propose a deep learning framework, Deep Side, for ADR prediction. Deep Side uses only (i) in vitro gene expression profiling experiments (GEX) and their experimental meta data (i.e., cell line and dosage - META), and (ii) the chemical structure of the compounds (CS). Our models train on the full LINCS L1000 dataset and use the SIDER dataset as the ground truth for drug - ADR pair labels [13]. We experiment with five architectures: (i) a multi-layer perceptron (MLP), (ii) MLP with residual connections (Res MLP), (iii) multi-modal neural networks (MMNN. Concat and MMNN. Sum), (iv) multi-task neural network (MTNN), and finally, (v) SMILES convolutional neural network (SMILES Conv).

We present an extensive evaluation of the above-mentioned architectures and investigate the contribution of different features. Our experiments show that CS is a robust predictor of side effects. The base MLP model, which uses CS features as input, produces $\sim 11\%$ macro-AUC and $\sim 2\%$ micro-AUC improvement over the state-of-the-art results provided in [32], which uses both GEX (high quality) and CS features. The multi-modal neural network model, which uses CS, GEX and META features and uses summation in the fusion layer (MMNN. Sum) achieves 0.79 macro-AUC and 0.877 micro-AUC which is the best result among MLP based approaches. We also find out that when the chemical structure features are fully utilized in a complex model like ours, it overpowers the information that is obtained from the GEX dataset. The convolutional neural network that only uses the SMILES string representation of the drug structures achieves the best result among all the proposed architectures with provides 13.0% macro-AUC and 3.1% micro-AUC improvement over the state-of-the-art algorithm. Finally, inspecting the confident false positives predictions reveal side effects that are not reported in the ground truth dataset, but are indeed reported in the literature.

CHAPTER-2

LITERATURE SURVEY

TITLE: DeepSide: A Deep Learning Approach for Drug Side Effect Prediction

AUTHOR: Onur Can Uner; Halil Ibrahim Kuru; R. Gokberk Cinbis; Oznur Tastan; A. Ercument Cicek

ABSTRACT: Drug failures due to unforeseen adverse effects at clinical trials pose health risks for the participants and lead to substantial financial losses. Side effect prediction algorithms have the potential to guide the drug design process. LINCS L1000 dataset provides a vast resource of cell line gene expression data perturbed by different drugs and creates a knowledge base for context specific features. The state-of-the-art approach that aims at using context specific information relies on only the high-quality experiments in LINCS L1000 and discards a large portion of the experiments. In this study, our goal is to boost the prediction performance by utilizing this data to its full extent. We experiment with 5 deep learning architectures. We find that a multi-modal architecture produces the best predictive performance among multi-layer perceptron-based architectures when drug chemical structure (CS), and the full set of drug perturbed gene expression profiles (GEX) are used as modalities. Overall, we observe that the CS is more informative than the GEX. A convolutional neural network-based model that uses only SMILES string representation of the drugs achieves the best results and provides 13.0% macro-AUC and 3.1% micro-AUC improvements over the state-of-the-art. We also show that the model is able to predict side effect-drug pairs that are reported in the literature but was missing in the ground truth side effect dataset.

TITLE: A Deep Learning Approach to the Prediction of Drug Side-Effects on Molecular Graphs

AUTHOR: Pietro Bongini, Elisa Messori, Monica Bianchini, Niccolò Pancino

ABSTRACT: Predicting drug side effects before they occur is a critical task for keeping the number of drug-related hospitalizations low and for improving drug discovery processes. Automatic predictors of side-effects generally are not able to process the structure of the drug, resulting in a loss of information. Graph neural networks have seen great success in recent years, thanks to their ability of exploiting the information conveyed by the graph structure and labels. These models have been used in a wide variety

of biological applications, among which the prediction of drug side-effects on a large knowledge graph. Exploiting the molecular graph encoding the structure of the drug represents a novel approach, in which the problem is formulated as a multi-class multi-label graph-focused classification. We developed a methodology to carry out this task, using recurrent Graph Neural Networks, and building a dataset from freely accessible and well established data sources. The results show that our method has an improved classification capability, under many parameters and metrics, with respect to previously available predictors. The method is not ready for clinical tests yet, as the specificity is still below the preliminary 25% threshold. Future efforts will aim at improving this aspect.

TITLE: Modular Multi-Source Prediction of Drug Side-Effects With DruGNN

AUTHOR: Pietro Bongini , Franco Scarselli , Monica Bianchini , Giovanna Maria Dimitri, Niccolo Pancino, and Pietro Li

ABSTRACT: —Drug Side-Effects (DSEs) have a high impact on public health, care system costs, and drug discovery processes. Predicting the probability of side-effects, before their occurrence, is fundamental to reduce this impact, in particular on drug discovery. Candidate molecules could be screened before undergoing clinical trials, reducing the costs in time, money, and health of the participants. Drug side-effects are triggered by complex biological processes involving many different entities, from drug structures to protein-protein interactions. To predict their occurrence, it is necessary to integrate data from heterogeneous sources. In this work, such heterogeneous data is integrated into a graph dataset, expressively representing the relational information between different entities, such as drug molecules and genes. The relational nature of the dataset represents an important novelty for drug side-effect predictors. Graph Neural Networks (GNNs) are exploited to predict DSEs on our dataset with very promising results. GNNs are deep learning models that can process graph-structured data, with minimal information loss, and have been applied on a wide variety of biological tasks. Our experimental results confirm the advantage of using relationships between data entities, suggesting interesting future developments in this scope. The experimentation also shows the importance of specific subsets of data in determining associations between drugs and side-effects

TITLE: Drug Side-Effect Prediction Based on the Integration of Chemical and Biological Spaces

AUTHOR: Yoshihiro Yamanishi^{*†}, Edouard Pauwels^{§‡}, and Masaaki Kotera^{//}

ABSTRACT: Drug side-effects, or adverse drug reactions, have become a major public health concern and remain one of the main causes of drug failure and of drug withdrawal once they have reached the market. Therefore, the identification of potential severe side-effects is a challenging issue. In this paper, we develop a new method to predict potential side-effect profiles of drug candidate molecules based on their chemical structures and target protein information on a large scale. We propose several extensions of kernel regression model for multiple responses to deal with heterogeneous data sources. The originality lies in the integration of the chemical space of drug chemical structures and the biological space of drug target proteins in a unified framework. As a result, we demonstrate the usefulness of the proposed method on the simultaneous prediction of 969 side-effects for approved drugs from their chemical substructure and target protein profiles and show that the prediction accuracy consistently improves owing to the proposed regression model and integration of chemical and biological information. We also conduct a comprehensive side-effect prediction for uncharacterized drug molecules stored in DrugBank and confirm interesting predictions using independent information sources. The proposed method is expected to be useful at many stages of the drug development process.

TITLE: Binary Relevance and MLSMOTE Based Computational Framework to Predict Drug Functions From Chemical and Biological Properties of Drug

AUTHOR: Pranab DasYogita Thakran,S. R. Ngamwal Anal,Vipin Pal,Anju Yadav,

ABSTRACT:In silico machine learning based prediction of drug functions considering the drug properties would substantially enhance the speed and reduce the cost of identifying promising drug leads. The drug function prediction capability of different drug properties happens to be different. So assessing these is advantageous in drug discovery. The task of drug function prediction is multi-label in nature reason being, in case of several drugs, multiple functions are associated with a drug. A number of existing works have ignored this inherent multi-label nature of the problem in context of addressing the issue of class imbalance. In the present work, a computational framework named as BRMCF has been proposed for analysing the prediction capability of chemical and biological properties of drugs

toward drug functions in view of multi-label nature of problem. It employs Binary Relevance (BR) approach along with five base classifiers for handling the multi-label prediction task and MLSMOTE for addressing the issue of class imbalance. The proposed framework has been validated and compared with BR, Classifier Chains (CC) and Deep Neural Network (DNN) method on four drug properties datasets: SMILES Strings (SS) dataset, 17 Molecular Descriptors (17MD) dataset, Protein Sequences (PS) dataset and drug perturbed Gene EXpression Profiles (GEX) dataset. The analysis of results shows that the proposed framework BRMCF has outperformed BR, CC and DNN method in terms of exact match ratio, precision, recall, F1-score, ROC-AUC which signifies the effectiveness of MLSMOTE. Further, assessment of prediction capability of different drug properties is done and they are ranked as SS >> GEX >> PS >> 17MD. Additionally, the visualization and analysis of drug function co-occurrences signify the appropriateness of the proposed framework for drug function co-occurrence detection and in signaling the new possible drug leads where the detection rate varies from 94.34% to 99.61%.

CHAPTER-3

SYSTEM ANALYSIS

3.1 EXISTING SYSTEM

A drug-drug interaction (DDI) is defined as an association between two drugs where the pharmacological effects of a drug are influenced by another drug. Positive DDIs can usually improve the therapeutic effects of patients, but negative DDIs cause the major cause of adverse drug reactions and even result in the drug withdrawal from the market and the patient death. Therefore, identifying DDIs has become a key component of the drug development and disease treatment.

In this study, an existing system, develops a method to predict DDIs based on the integrated similarity and semi-supervised learning (DDI-IS-SL). DDI-IS-SL integrates the drug chemical, biological and phenotype data to calculate the feature similarity of drugs with the cosine similarity method. The Gaussian Interaction Profile kernel similarity of drugs is also calculated based on known DDIs. A semi-supervised learning method (the Regularized Least Squares classifier) is used to calculate the interaction possibility scores of drug-drug pairs. In terms of the 5-fold cross validation, 10-fold cross validation and de novo drug validation, DDI-IS-SL can achieve the better prediction performance than other comparative methods. In addition, the average computation time of DDI-IS-SL is shorter than that of other comparative methods. Finally, case studies further demonstrate the performance of DDI-IS-SL in practical applications.

DISADVANTAGES

- The complexity of data: Most of the existing machine learning models must be able to accurately interpret large and complex datasets to detect an accurate Drug Side Effect.
- Data availability: Most machine learning models require large amounts of data to create accurate predictions. If data is unavailable in sufficient quantities, then model accuracy may suffer.
- Incorrect labeling: The existing machine learning models are only as accurate as the data trained using the input dataset. If the data has been incorrectly labeled, the model cannot make accurate predictions.

3.2 PROPOSED SYSTEM

Multi-layer perceptron (MLP) Our MLP [22] model takes the concatenation of all input vectors and applies a series of fully-connected (FC) layers. Each FC layer is followed by a batch normalization layer [10]. We use ReLU activation [16], and dropout regularization [27] with a drop probability of 0.2. The sigmoid activation function is applied to the final layer outputs, which yields the ADR prediction probabilities. The loss function is defined as the sum of negative log- probabilities over ADR classes, i.e. the multi-label binary cross-entropy loss (BCE). An illustration of the architecture for CS and GEX features is given in this system.

Residual multi-layer perceptron (ResMLP) The residual multi-layer perceptron (ResMLP) architecture is very similar to MLP, except that it uses residual-connections across the fully- connected layers. More specifically, the input of each intermediate layer is element-wise added to its output, before getting processed by the next layer. Such residual connections have been shown to reduce the vanishing gradient problem to a large extent [7].

This effectively allows deeper architectures, therefore, potentially learning more complex and parameter-efficient feature extractors. **Multi-modal neural networks (MMNN)** The multi-modal neural network approach contains distinct MLP sub-networks where each one extract features from one data modality only. The outputs of these sub-networks are then fused and fed to the classification block. For feature fusion, we consider two strategies: concatenation and summation. While the former one concatenates the domain-specific feature vectors to a larger one, the latter one performs element-wise summation. By definition, for summation based fusion, the domain-specific feature extraction sub-networks have to be designed to produce vectors of equivalent sizes. We refer to the concatenation and summation based MMNN networks as MMNN.Concat and MMNN.Sum, respectively.

Multi-task neural network (MTNN) our multitask learning (MTL) based architecture aims to take the side effect groups obtained from the taxonomy of ADReCS into account. For this purpose, the approach defines shared and task-specific MLP sub-network blocks. The shared block takes the concatenation of GEX and CS features as input and outputs a joint embedding. Each task-specific sub-network then converts the joint embedding into a vector of binary prediction scores for a set of inter-related side-effect classes.

ADVANTAGES

- The proposed system implemented many ml classifies for testing and training on datasets.
- The proposed system developed Convolutional neural networks(CNN) which are known to provide a powerful way of automatically learning complex features in vision tasks to find an accurate accuracy on the datasets.

CHAPTER-4

SYSTEM REQUIREMENTS

4.1 FUNCTIONAL REQUIREMENTS

Enabler moduler:In this module, the Enable user has to login by using valid user name and password. After login successful he can do some operations such as Train & Test drug Data Sets, View Trained and Tested Accuracy in Bar Chart, View Trained and Tested Accuracy Results,view trained and tested drug datasets accuracy results, view drug side effect prediction type.

External client moduler:In this module, there are n numbers of users are present. User should register before doing any operations. Once user registers, their details will be stored to the database. After registration successful, he has to login by using authorized user name and password.

4.2 NON-FUNCTIONAL REQUIREMENTS

4.2.1 HARDWARE REQUIREMENTS

- Processor - Pentium –IV
- RAM - 4 GB (min)
- Hard Disk - 20 GB
- Key Board - Standard Windows Keyboard
- Mouse - Two or Three Button Mouse
- Monitor - SVGA

4.2.2 SOFTWARE REQUIREMENTS

- ❖ Operating system : Windows 7 Ultimate.
- ❖ Coding Language : Python.
- ❖ Front-End : Python.
- ❖ Back-End : Django-ORM
- ❖ Designing : Html, css, javascript.
- ❖ Data Base : MySQL (WAMP Server).

CHAPTER-5

SYSTEM STUDY

5.1 FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

5.2FEASIBILITYANALYSIS

Three key considerations involved in the feasibility analysis are

- ◆ ECONOMICAL FEASIBILITY
- ◆ TECHNICAL FEASIBILITY
- ◆ SOCIAL FEASIBILITY

ECONOMICAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

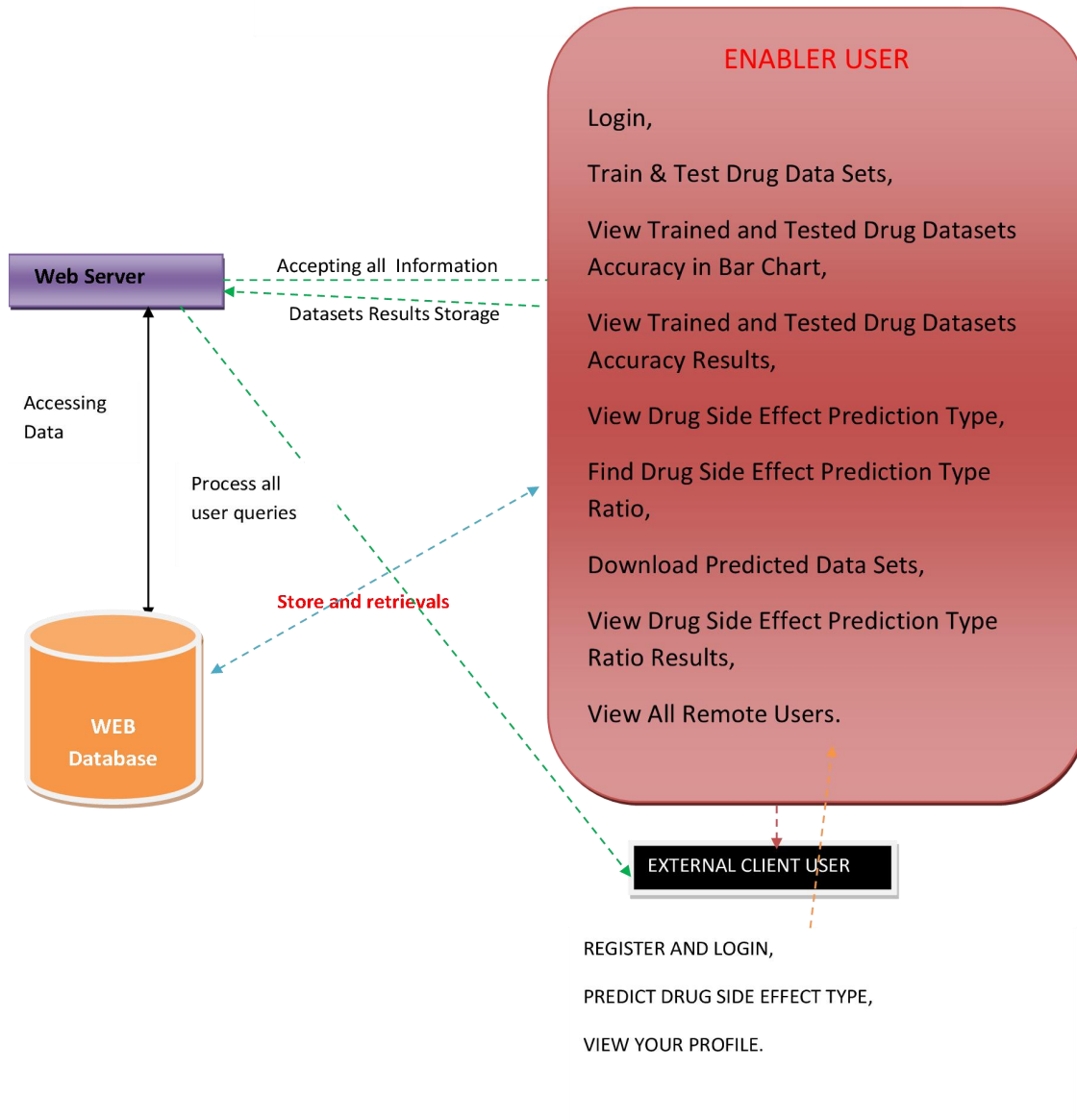
SOCIAL FEASIBILITY

The aspect of study is to check the level of acceptance of the system by the user. This includes the process of training the user to use the system efficiently. The user must not feel threatened by the system, instead must accept it as a necessity. The level of acceptance by the users solely depends on the methods that are employed to educate the user about the system and to make him familiar with it. His level of confidence must be raised so that he is also able to make some constructive criticism, which is welcomed, as he is the final user of the system

CHAPTER-6

SYSTEM DESIGN

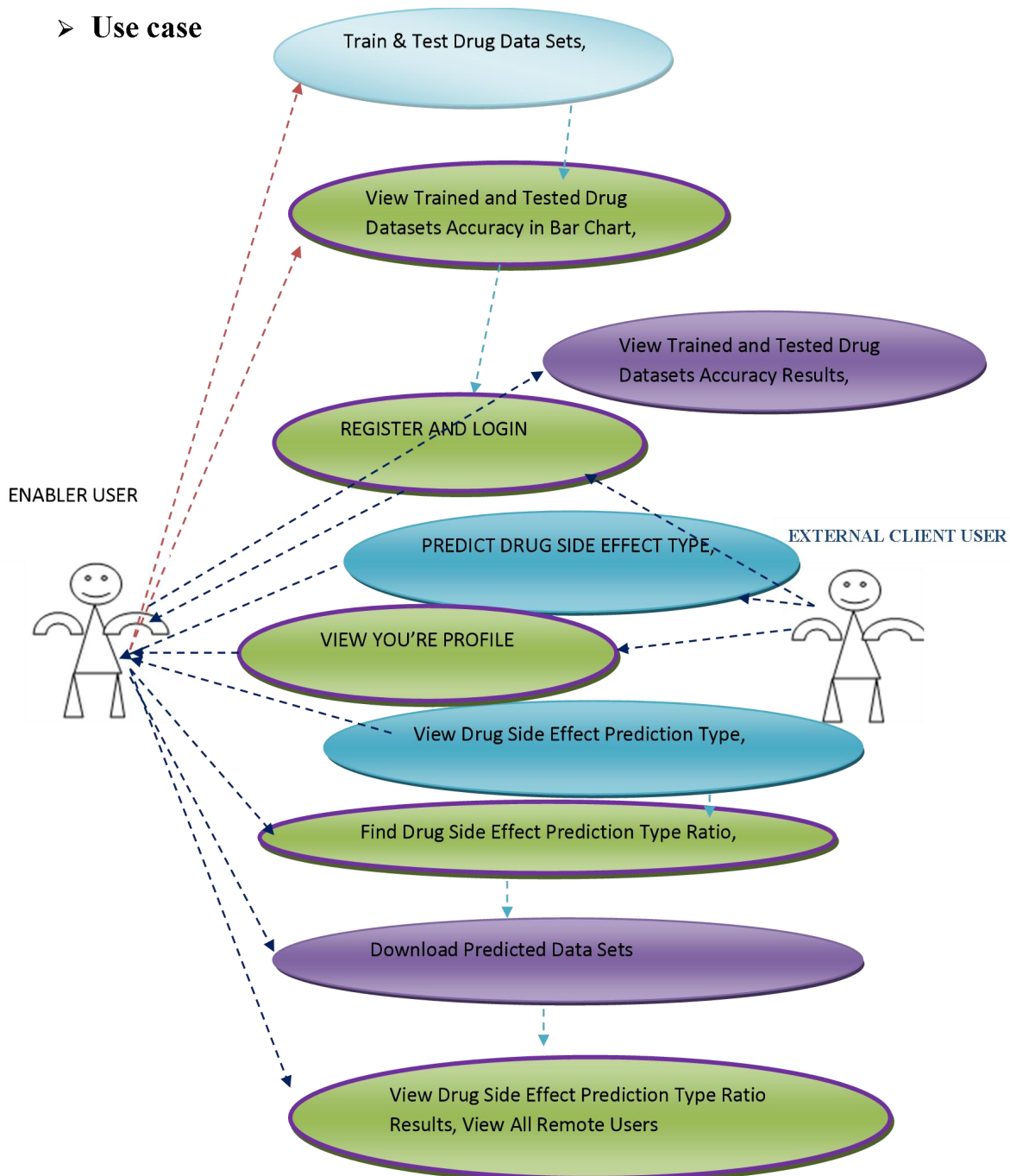
6.1 SYSTEM ARCHITECTURE



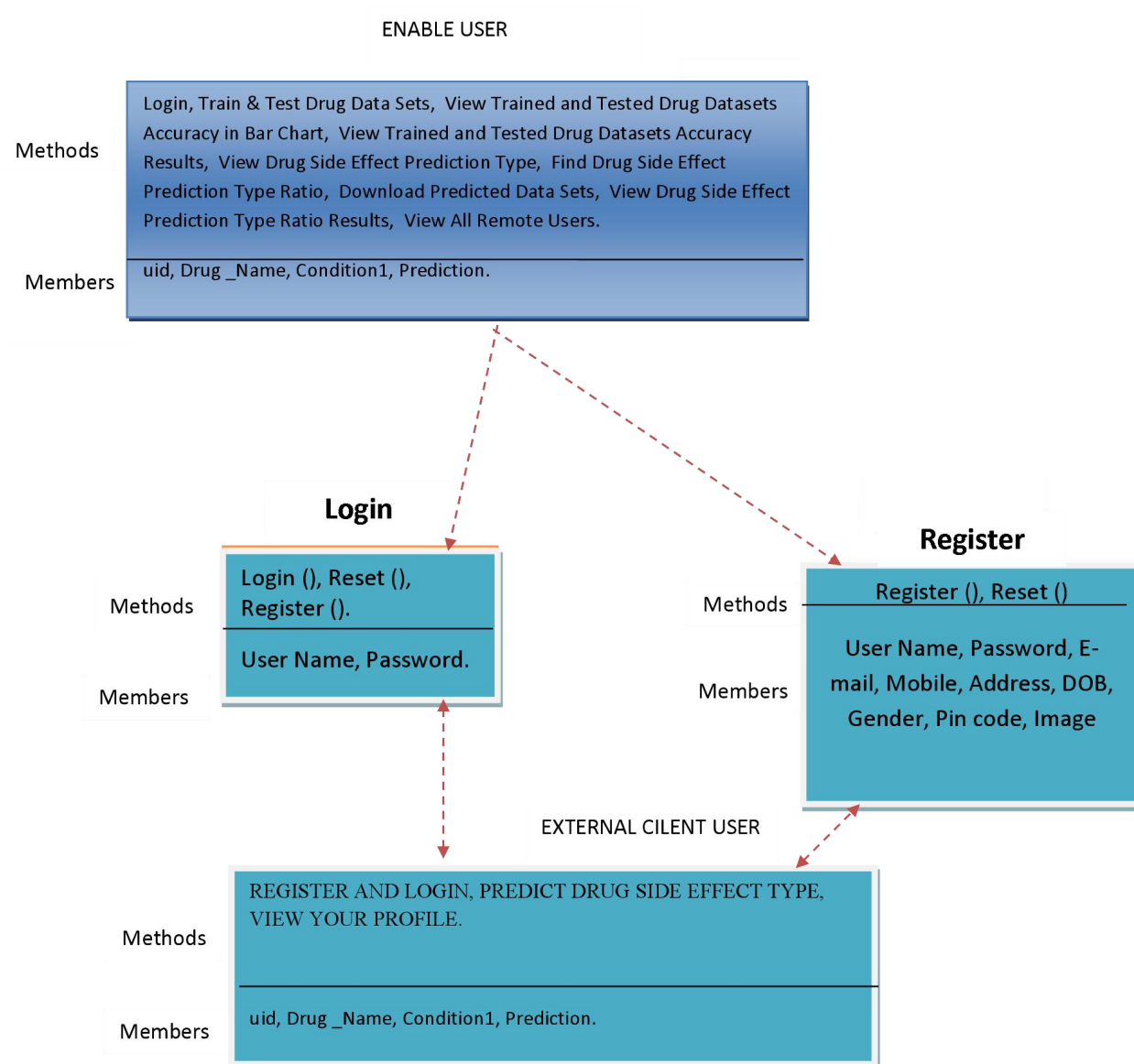
6.2 UML DIAGRAMS

6.2.1 USECASE DIAGRAM

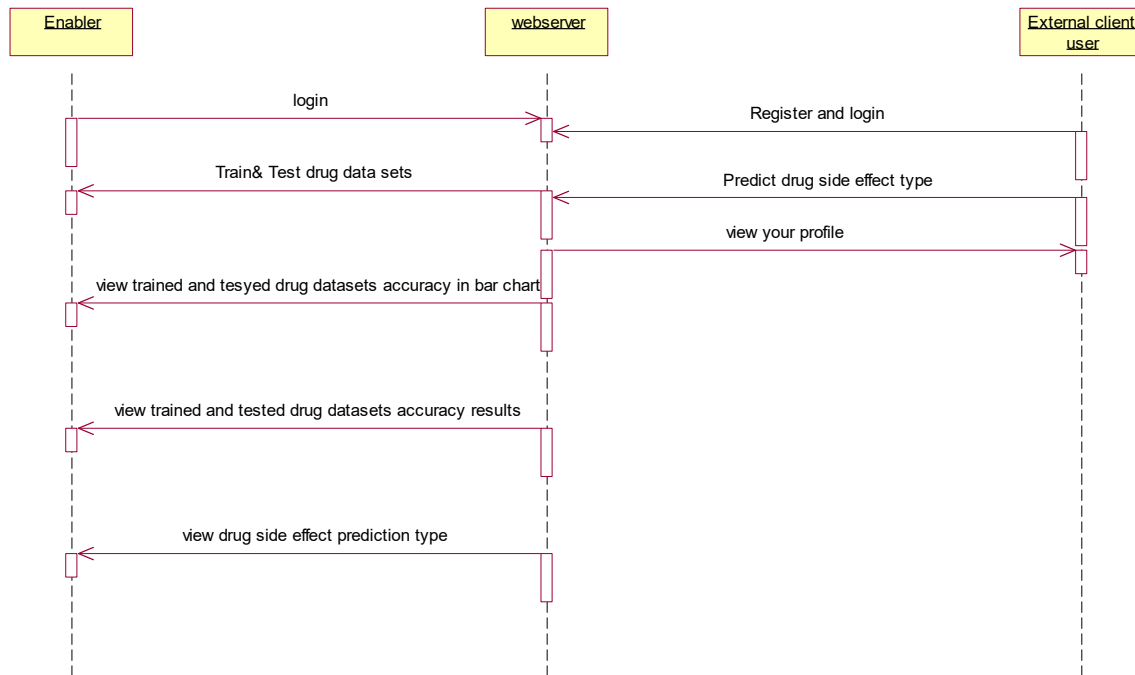
➤ Use case



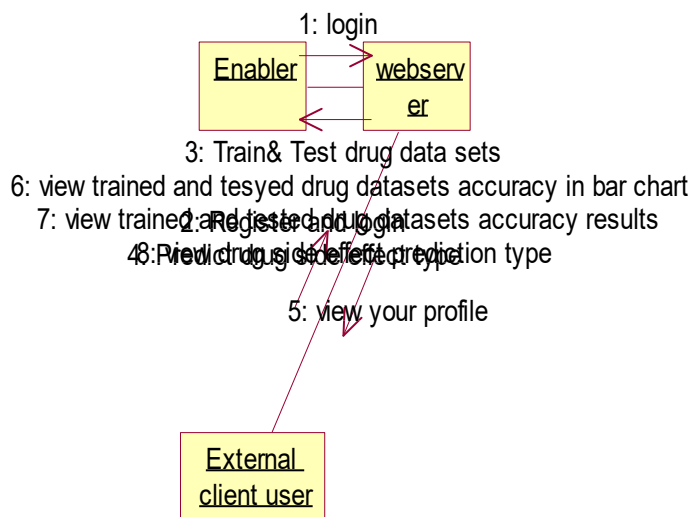
6.2.2 CLASS DIAGRAM



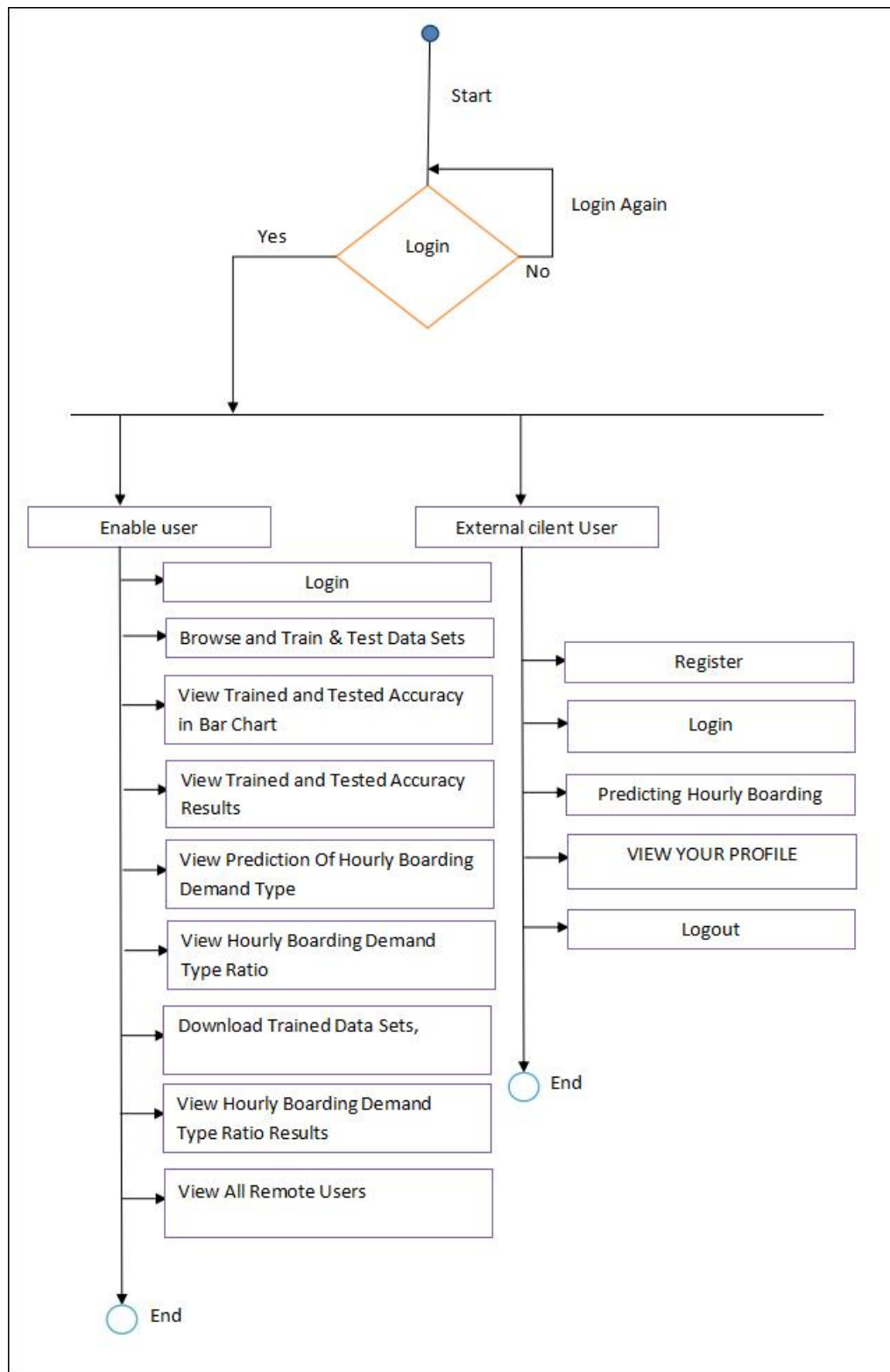
6.2.3 SEQUENCE DIAGRAM



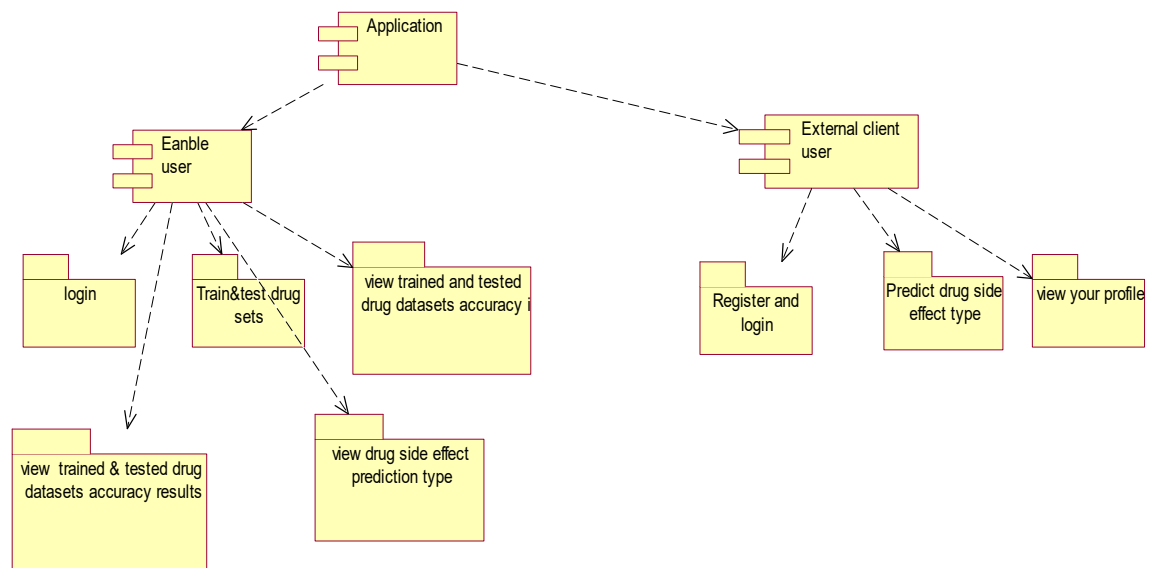
6.2.4 COLLABRATION DIAGRAM



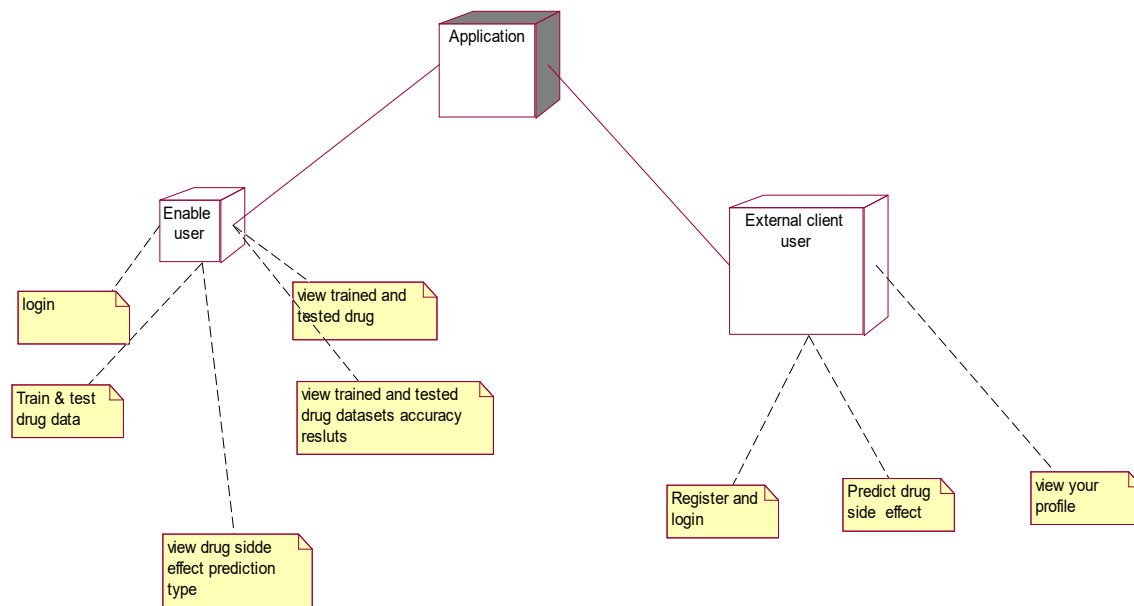
6.2.5 ACTIVITY DIAGRAM



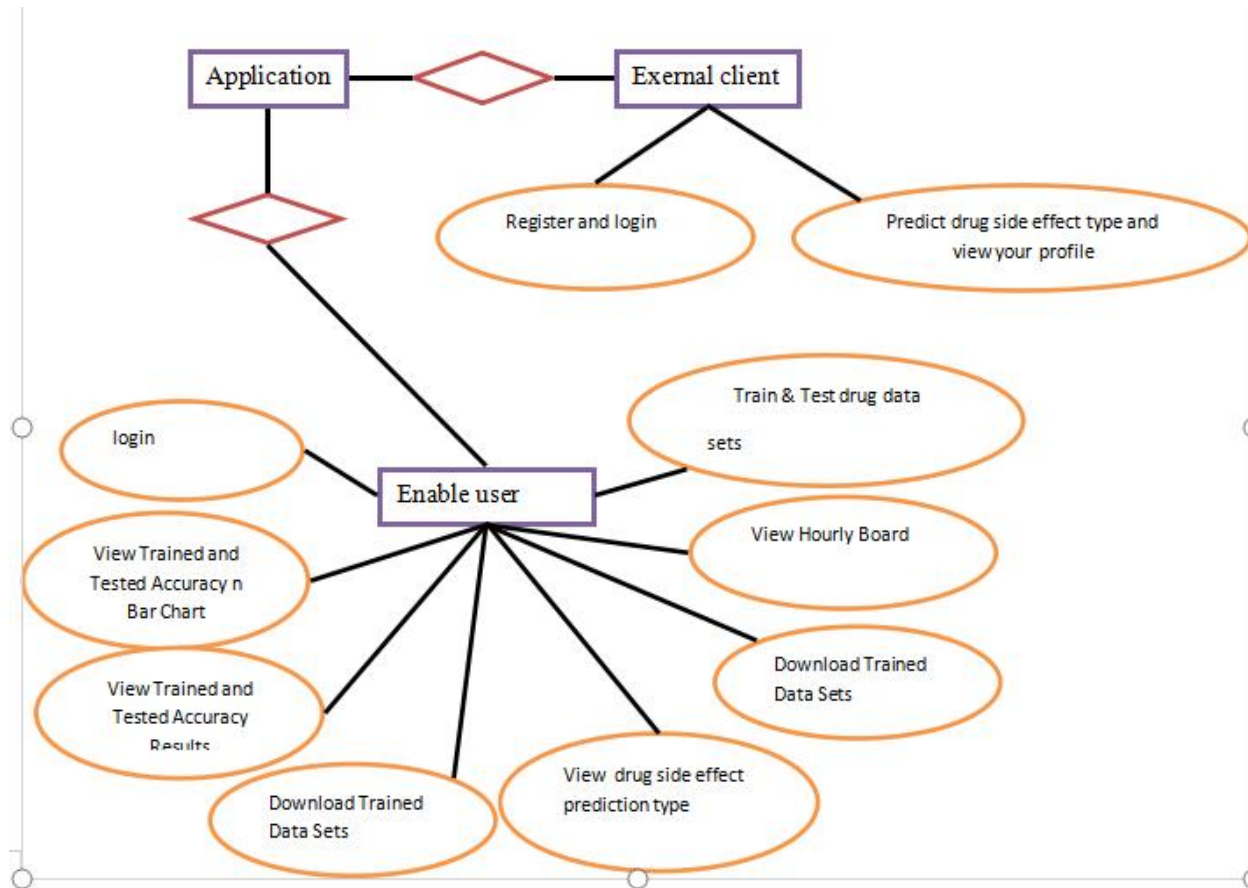
6.2.6 COMPONENT DIAGRAM



6.2.7 DEPLOYMENT DIAGRAM



6.2.8 ER DIAGRAM



6.2.9 DATA DICTIONARY

Database: Deep Side A Deep Learning Framework for Drug Side Effect Prediction

Table name: auth_group

Column	Data Type	Constraints	Description
id	Int(11)	Primary key	Unique Identifier
name	Varchar(1000)	Not null	name

Column	Data Type	Constraints	Description
id	Int(11)	Primary key	Unique Identifier

Group id	Int(11)	Primary Key	Unique Identifier	Table Name: Auth_
Permission_id	Int(11)	Primary Key	Unique Identifier	

group_Permission

Table Name: auth_permission

Column	Data Type	Constraints	Description
id	Int(11)	Primary key	Unique Identifier
Name	Int(255)	Not Null	name
Content_type_id	Int(11)	Primary Key	Unique Identifier
Code name	Int(100)	Not Null	name

Table Name: auth_User

Column	Data Type	Constraints	Description
id	Int(11)	Primary key	Unique Identifier
Password	varchar(128)	Not Null	password
Last_Login	Datetime(6)	Not Null	Last login
Is_superuser	tinyint(1)	Not Null	Name
username	Varchar(150)	Not Null	Username
lastname	Varchar(30)	Not Null	Lastname
email	Varchar(150)	Not Null	Email id
Is_staff	Tinyint(1)	Not Null	Staff
Is_active	Tinyint(1)	Not Null	Active

Date_joined	Datetime(6)	Not Null	Date and time
-------------	-------------	----------	---------------

Table Name: auth_user_groups

Column	Data Type	Constraints	Description
id	Int(11)	Primary key	Unique Identifier
User_id	Int(11)	Primary Key	Unique Identifier
Group_id	Int(11)	Primary Key	Unique Identifier

CHAPTER-7

INPUT AND OUTPUT DESIGN

7.1 INPUT DESIGN

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data in to a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use with retaining the privacy. Input Design considered the following things:

- What data should be given as input?
- How the data should be arranged or coded?
- The dialog to guide the operating personnel in providing input.
- Methods for preparing input validations and steps to follow when error occur.

7.1.1 OBJECTIVES

1. Input Design is the process of converting a user-oriented description of the input into a computer-based system. This design is important to avoid errors in the data input process and show the correct direction to the management for getting correct information from the computerized system.

2. It is achieved by creating user-friendly screens for the data entry to handle large volume of data. The goal of designing input is to make data entry easier and to be free from errors. The data entry screen is designed in such a way that all the data manipulates can be performed. It also provides record viewing facilities.

3. When the data is entered it will check for its validity. Data can be entered with the help of screens. Appropriate messages are provided as when needed so that the user will not be in maize of instant. Thus the objective of input design is to create an input layout that is easy to follow.

7.2 OUTPUT DESIGN

A quality output is one, which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other system through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the most important and direct source information to the user. Efficient and intelligent output design improves the system's relationship to help user decision-making.

1. Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will find the system can use easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements.

2. Select methods for presenting information.

3. Create document, report, or other formats that contain information produced by the system.

The output form of an information system should accomplish one or more of the following objectives.

- Convey information about past activities, current status or projections of the
- Future.
- Signal important events, opportunities, problems, or warnings.
- Trigger an action.
- Confirm an action.

CHAPTER-8

IMPLEMENTATION

8.1 MODULES

- Enable user
- External client user

8.1.1 MODULE DESCRIPTION

ENABLE USER: In this module, the Enable user has to login by using valid user name and password. After login successful he can do some operations such as Train & Test drug Data Sets, View Trained and Tested Accuracy in Bar Chart, View Trained and Tested Accuracy Results, view trained and tested drug datasets accuracy results, view drug side effect prediction type.

View and Authorize Users

In this module, the admin can view the list of users who all registered. In this, the admin can view the user's details such as, user name, email, address and admin authorizes the users.

EXTERNAL CLIENT USER:

In this module, there are n numbers of users are present. User should register before doing any operations. Once user registers, their details will be stored to the database. After registration successful, he has to login by using authorized user name and password. Once Login is successful user will do some operations like REGISTER AND LOGIN, PREDICT DRUG SIDE EFFECT TYPE, VIEW YOUR PROFILE.

CHAPTER-9

SOFTWARE ENVIRONMENT

9.1 PYTHON

Python is a **high-level, interpreted, interactive and object-oriented scripting language**. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

- **Python is Interpreted:** Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive:** You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented:** Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python is a Beginner's Language:** Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

History of Python

Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands.

Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol-68, SmallTalk, and Unix shell and other scripting languages.

Python is copyrighted. Like Perl, Python source code is now available under the GNU General Public License (GPL).

Python is now maintained by a core development team at the institute, although Guido van Rossum still holds a vital role in directing its progress.

Python Features

Python's features include:

- **Easy-to-learn:** Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- **Easy-to-read:** Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain:** Python's source code is fairly easy-to-maintain.
- **A broad standard library:** Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.
- **Interactive Mode:** Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **Portable:** Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable:** You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Databases:** Python provides interfaces to all major commercial databases.
- **GUI Programming:** Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- **Scalable:** Python provides a better structure and support for large programs than shell scripting.

Python has a big list of good features:

- It supports functional and structured programming methods as well as OOP.
- It can be used as a scripting language or can be compiled to byte-code for building large applications.
- It provides very high-level dynamic data types and supports dynamic type checking.
- IT supports automatic garbage collection.
- It can be easily integrated with C, C++, COM, ActiveX, CORBA, and Java.



ARITHMETIC OPERATORS

Operator	Description	Example
+ Addition	Adds values on either side of the operator.	$a + b = 30$
- Subtraction	Subtracts right hand operand from left hand operand.	$a - b = -10$
* Multiplication	Multiplies values on either side of the operator	$a * b = 200$
/ Division	Divides left hand operand by right hand operand	$b / a = 2$
% Modulus	Divides left hand operand by right hand operand and returns remainder	$b \% a = 0$
** Exponent	Performs exponential (power) calculation on operators	$a ** b = 10$ to the power 20
//	Floor Division - The division of operands where the result is the quotient in which the digits after the	$9 // 2 = 4$ and

	decimal point are removed. But if one of the operands is negative, the result is floored, i.e., rounded away from zero (towards negative infinity):	$9.0//2.0 = 4.0$, - $11//3 = -4$, - $11.0//3 = -4.0$
--	---	--

ASSIGNMENT OPERATOR

Operator	Description	Example
=	Assigns values from right side operands to left side operand	$c = a + b$ assigns value of $a + b$ into c
+= Add AND	It adds right operand to the left operand and assign the result to left operand	$c += a$ is equivalent to $c = c + a$
-= Subtract AND	It subtracts right operand from the left operand and assign the result to left operand	$c -= a$ is equivalent to $c = c -$

		a
<code>*=</code> Multiply AND	It multiplies right operand with the left operand and assign the result to left operand	<code>c *= a</code> is equivalent to <code>c = c * a</code>
<code>/=</code> Divide AND	It divides left operand with the right operand and assign the result to left operand	<code>c /= a</code> is equivalent to <code>c = c / a</code> <code>ac /= a</code> is equivalent to <code>c = c / a</code>

<code>%=</code> Modulus AND	It takes modulus using two operands and assign the result to left operand	<code>c %= a</code> is equivalent to <code>c = c % a</code>
<code>**=</code> Exponent	Performs exponential (power) calculation on operators and assign value to the left	<code>c **= a</code> is equivalent to <code>c = c ** a</code>

AND	operand	** a
//= Floor Division	It performs floor division on operators and assign value to the left operand	c //= a is equivalent to c = c // a

IDENTITY OPERATOR

Operator	Description	Example
is	Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.	x is y, here is results in 1 if id(x) equals id(y).
is not	Evaluates to false if the variables on either side of the operator point to the same object and true otherwise.	x is not y, here is not results in 1 if id(x) is not equal to id(y)

COMPARISON OPERATOR

Operator	Description	Example
& Binary AND	Operator copies a bit to the result if it exists in both operands	(a & b) (means 0000 1100)
Binary OR	It copies a bit if it exists in either operand.	(a b) = 61 (means 0011 1101)
^ Binary XOR	It copies the bit if it is set in one operand but not both.	(a ^ b) = 49 (means 0011 0001)
~ Binary Ones Complement	It is unary and has the effect of 'flipping' bits.	(~a) = -61 (means 1100 0011 in 2's complement form due to a signed binary number.
<< Binary Left Shift	The left operands value is moved left by the number of bits	a << 2 = 240 (means

	specified by the right operand.	1111 0000)
>> Binary Right Shift	The left operands value is moved right by the number of bits specified by the right operand.	a >> 2 = 15 (means 0000 1111)

LOGICAL OPERATOR

Operator	Description	Example
and Logical AND	If both the operands are true then condition becomes true.	(a and b) is true.
or Logical OR	If any of the two operands are non-zero then condition becomes true.	(a or b) is true.
not Logical NOT	Used to reverse the logical state of its operand.	Not(a and b) is false.

Membership Operators

Operator	Description	Example
in	Evaluates to true if it finds a variable in the specified sequence and false otherwise.	x in y, here in results in a 1 if x is a member of sequence y.
not in	Evaluates to true if it does not finds a variable in the specified sequence and false otherwise.	x not in y, here not in results in a 1 if x is not a member of sequence y.

Python Operators Precedence

Operator	Description
**	Exponentiation (raise to the power)
~ + -	Complement, unary plus and minus (method names for the last two are +@ and -@)
* / % //	Multiply, divide, modulo and floor division

+ -	Addition and subtraction
>> <<	Right and left bitwise shift
&	Bitwise 'AND'
^	Bitwise exclusive 'OR' and regular 'OR'
<= < > >=	Comparison operators
<> == !=	Equality operators
= %= /= //= -= += *= **=	Assignment operators
is is not	Identity operators
in not in	Membership operators
not or and	Logical operators

LIST

The list is a most versatile data type available in Python which can be written as a list of comma-separated values (items) between square brackets. Important thing about a list is that items in a list need not be of the same type.

Creating a list is as simple as putting different comma-separated values between square brackets. For example –


```
list1 = ['physics', 'chemistry', 1997, 2000];
```

```
list2 = [1, 2, 3, 4, 5];
```

```
list3 = ["a", "b", "c", "d"]
```

Basic List Operations

Lists respond to the + and * operators much like strings; they mean concatenation and repetition here too, except that the result is a new list, not a string.

Python Expression	Results	Description
len([1, 2, 3])	3	Length
[1, 2, 3] + [4, 5, 6]	[1, 2, 3, 4, 5, 6]	Concatenation
['Hi!'] * 4	['Hi!', 'Hi!', 'Hi!', 'Hi!']	Repetition
3 in [1, 2, 3]	True	Membership
for x in [1, 2, 3]: print x,	1 2 3	Iteration

Built-in List Functions & Methods:

Python includes the following list functions –

SN	Function with Description
----	---------------------------

1	<u>cmp(list1, list2)</u> Compares elements of both lists.
2	<u>len(list)</u> Gives the total length of the list.
3	<u>max(list)</u> Returns item from the list with max value.
4	<u>min(list)</u> Returns item from the list with min value.
5	<u>list(seq)</u> Converts a tuple into list.

Python includes following list methods

SN	Methods with Description
1	<u>list.append(obj)</u> Appends object obj to list
2	<u>list.count(obj)</u> Returns count of how many times obj occurs in list

3	<code>list.extend(seq)</code> Appends the contents of seq to list
4	<code>list.index(obj)</code> Returns the lowest index in list that obj appears
5	<code>list.insert(index, obj)</code> Inserts object obj into list at offset index
6	<code>list.pop(obj=list[-1])</code> Removes and returns last object or obj from list
7	<code>list.remove(obj)</code> Removes object obj from list
8	<code>list.reverse()</code> Reverses objects of list in place
9	<code>list.sort([func])</code> Sorts objects of list, use compare function if given

TUPLES

A tuple is a sequence of immutable Python objects. Tuples are sequences, just like lists. The differences between tuples and lists are, the tuples cannot be changed unlike lists and tuples use parentheses, whereas lists use square brackets.

Creating a tuple is as simple as putting different comma-separated values. Optionally we can put these comma-separated values between parentheses also. For example –

```
tup1 = ('physics', 'chemistry', 1997, 2000);  
  
tup2 = (1, 2, 3, 4, 5 );  
  
tup3 = "a", "b", "c", "d";
```

The empty tuple is written as two parentheses containing nothing –

```
tup1 = ();
```

To write a tuple containing a single value you have to include a comma, even though there is only one value –

```
tup1 = (50,);
```

Like string indices, tuple indices start at 0, and they can be sliced, concatenated, and so on.

- **Accessing Values in Tuples:**

To access values in tuple, use the square brackets for slicing along with the index or indices to obtain value available at that index. For example –

```
tup1 = ('physics', 'chemistry', 1997, 2000);  
  
tup2 = (1, 2, 3, 4, 5, 6, 7 );  
  
print "tup1[0]: ", tup1[0]  
  
print "tup2[1:5]: ", tup2[1:5]
```

When the code is executed, it produces the following result –

```
tup1[0]: physics  
tup2[1:5]: [2, 3, 4, 5]
```

Updating Tuples:

Tuples are immutable which means you cannot update or change the values of tuple elements. We are able to take portions of existing tuples to create new tuples as the following example demonstrates –

```
tup1 = (12, 34.56);  
tup2 = ('abc', 'xyz');  
tup3 = tup1 + tup2;  
print tup3
```

When the above code is executed, it produces the following result –

```
(12, 34.56, 'abc', 'xyz')
```

Delete Tuple Elements

Removing individual tuple elements is not possible. There is, of course, nothing wrong with putting together another tuple with the undesired elements discarded.

To explicitly remove an entire tuple, just use the **del** statement. For example:

```
tup = ('physics', 'chemistry', 1997, 2000);  
print tup  
del tup;  
print "After deleting tup : "  
print tup
```

Basic Tuples Operations:

Python Expression	Results	Description
<code>len((1, 2, 3))</code>	3	Length
<code>(1, 2, 3) + (4, 5, 6)</code>	<code>(1, 2, 3, 4, 5, 6)</code>	Concatenation
<code>('Hi!') * 4</code>	<code>('Hi!', 'Hi!', 'Hi!', 'Hi!')</code>	Repetition
<code>3 in (1, 2, 3)</code>	True	Membership
<code>for x in (1, 2, 3): print x,</code>	1 2 3	Iteration

Built-in Tuple Functions

SN	Function with Description
1	<code>cmp(tuple1, tuple2)</code> :Compares elements of both tuples.
2	<code>len(tuple)</code> :Gives the total length of the tuple.
3	<code>max(tuple)</code> :Returns item from the tuple with max value.
4	<code>min(tuple)</code> :Returns item from the tuple with min value.

5	tuple(seq) : Converts a list into tuple.
---	--

DICTIONARY

Each key is separated from its value by a colon (:), the items are separated by commas, and the whole thing is enclosed in curly braces. An empty dictionary without any items is written with just two curly braces, like this: {}.

Keys are unique within a dictionary while values may not be. The values of a dictionary can be of any type, but the keys must be of an immutable data type such as strings, numbers, or tuples.

Accessing Values in Dictionary:

To access dictionary elements, you can use the familiar square brackets along with the key to obtain its value. Following is a simple example –

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
```

```
print "dict['Name']: ", dict['Name']
```

```
print "dict['Age']: ", dict['Age']
```

Result –

```
dict['Name']: Zara
```

```
dict['Age']: 7
```

Updating Dictionary

We can update a dictionary by adding a new entry or a key-value pair, modifying an existing entry, or deleting an existing entry as shown below in the simple example –

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}

dict['Age'] = 8; # update existing entry

dict['School'] = "DPS School"; # Add new entry

print "dict['Age']: ", dict['Age']

print "dict['School']: ", dict['School']
```

Result –

```
dict['Age']: 8
dict['School']: DPS School
```

Delete Dictionary Elements

We can either remove individual dictionary elements or clear the entire contents of a dictionary. You can also delete entire dictionary in a single operation.

To explicitly remove an entire dictionary, just use the **del** statement. Following is a simple example –

```
dict = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}

del dict['Name']; # remove entry with key 'Name'

dict.clear();    # remove all entries in dict

del dict ;      # delete entire dictionary

print "dict['Age']: ", dict['Age']
```



```
print "dict['School']: ", dict['School']
```

Built-in Dictionary Functions & Methods –

Python includes the following dictionary functions –

SN	Function with Description
1	cmp(dict1, dict2) Compares elements of both dict.
2	len(dict) Gives the total length of the dictionary. This would be equal to the number of items in the dictionary.
3	str(dict) Produces a printable string representation of a dictionary
4	type(variable) Returns the type of the passed variable. If passed variable is dictionary, then it would return a dictionary type.

Python includes following dictionary methods –

SN	Methods with Description
1	<code>dict.clear()</code> :Removes all elements of dictionary <i>dict</i>
2	<code>dict.Copy()</code> :Returns a shallow copy of dictionary <i>dict</i>
3	<code>dict.fromkeys()</code> :Create a new dictionary with keys from seq and values <i>set</i> to <i>value</i> .
4	<code>dict.get(key, default=None)</code> :For <i>key</i> key, returns value or default if key not in dictionary
5	<code>dict.has_key(key)</code> :Returns <i>true</i> if key in dictionary <i>dict</i> , <i>false</i> otherwise
6	<code>dict.items()</code> :Returns a list of <i>dict</i> 's (key, value) tuple pairs
7	<code>dict.keys()</code> :Returns list of dictionary <i>dict</i> 's keys
8	<code>dict.setdefault(key, default=None)</code> :Similar to <code>get()</code> , but will set <code>dict[key]=default</code> if <i>key</i> is not already in <i>dict</i>
9	<code>dict.update(dict2)</code> :Adds dictionary <i>dict2</i> 's key-values pairs to <i>dict</i>
10	<code>dict.values()</code> :Returns list of dictionary <i>dict</i> 's values

A function is a block of organized, reusable code that is used to perform a single, related action. Functions provide better modularity for your application and a high degree of code reusing. Python gives you many built-in functions like `print()`, etc. but you can also create your own functions. These functions are called *user-defined functions*.

Defining a Function

Simple rules to define a function in Python.

- Function blocks begin with the keyword `def` followed by the function name and parentheses `( )`.
- Any input parameters or arguments should be placed within these parentheses. You can also define parameters inside these parentheses.
- The first statement of a function can be an optional statement - the documentation string of the function or *docstring*.
- The code block within every function starts with a colon `(:)` and is indented.
- The statement `return [expression]` exits a function, optionally passing back an expression to the caller. A return statement with no arguments is the same as `return None`.

```
def functionname( parameters ):

    "function_docstring"

    function_suite

    return [expression]
```

Calling a Function

Defining a function only gives it a name, specifies the parameters that are to be included in the function and structures the blocks of code. Once the basic structure of a function is finalized,

you can execute it by calling it from another function or directly from the Python prompt. Following is the example to call printme() function –

```
# Function definition is here

def printme( str ):

    "This prints a passed string into this function"

    print str

    return;

# Now you can call printme function

printme("I'm first call to user defined function!")

printme("Again second call to the same function")
```

When the above code is executed, it produces the following result –

```
I'm first call to user defined function!

Again second call to the same function
```

Function Arguments

You can call a function by using the following types of formal arguments:

- Required arguments
- Keyword arguments
- Default arguments
- Variable-length arguments

Scope of Variables

All variables in a program may not be accessible at all locations in that program. This depends on where you have declared a variable.

The scope of a variable determines the portion of the program where you can access a particular identifier. There are two basic scopes of variables in Python –

Global variables

Local variables

Global vs. Local variables

Variables that are defined inside a function body have a local scope, and those defined outside have a global scope.

This means that local variables can be accessed only inside the function in which they are declared, whereas global variables can be accessed throughout the program body by all functions. When you call a function, the variables declared inside it are brought into scope. Following is a simple example.

```
total = 0; # This is global variable.

# Function definition is here

def sum( arg1, arg2 ):

    # Add both the parameters and return them."

    total = arg1 + arg2; # Here total is local variable.

    print "Inside the function local total : ", total

    return total;

sum( 10, 20 );

print "Outside the function global total : ", total
```

Result –

Inside the function local total : 30

Outside the function global total : 0

A module allows you to logically organize your Python code. Grouping related code into a module makes the code easier to understand and use. A module is a Python object with arbitrarily named attributes that you can bind and reference. Simply, a module is a file consisting of Python code. A module can define functions, classes and variables. A module can also include runnable code.

Example:

The Python code for a module named *aname* normally resides in a file named *aname.py*. Here's an example of a simple module, support.py

```
def print_func( par ):
    print "Hello : ", par
    return
```

The *import* Statement

The *import* has the following syntax:

```
import module1[, module2[,... moduleN]
```

When the interpreter encounters an import statement, it imports the module if the module is present in the search path. A search path is a list of directories that the interpreter searches before importing a module. For example, to import the module support.py, you need to put the following command at the top of the script –

A module is loaded only once, regardless of the number of times it is imported. This prevents the module execution from happening over and over again if multiple imports occur.

Packages in Python

A package is a hierarchical file directory structure that defines a single Python application environment that consists of modules and sub packages and sub-sub packages.

Consider a file *Pots.py* available in *Phone* directory. This file has following line of source code

—

```
def Pots():  
  
    print "I'm Pots Phone"
```

Similar way, we have another two files having different functions with the same name as above

—

- *Phone/Isdn.py* file having function *Isdn()*
- *Phone/G3.py* file having function *G3()*

Now, create one more file *__init__.py* in *Phone* directory —

- *Phone/__init__.py*
- To make all of your functions available when you've imported *Phone*, to put explicit import statements in *__init__.py* as follows —

```
from Pots import Pots  
  
from Isdn import Isdn  
  
from G3 import G3
```

After you add these lines to *__init__.py*, you have all of these classes available when you import the *Phone* package.

```
# Now import your Phone Package.
```

```
import Phone

Phone.Pots()

Phone.Isdn()

Phone.G3()
```

RESULT:

```
I'm Pots Phone

I'm 3G Phone

I'm ISDN Phone
```

In the above example, we have taken example of a single functions in each file, but you can keep multiple functions in your files. You can also define different Python classes in those files and then you can create your packages out of those classes.

This chapter covers all the basic I/O functions available in Python.

Printing to the Screen

The simplest way to produce output is using the *print* statement where you can pass zero or more expressions separated by commas. This function converts the expressions you pass into a string and

writes the result to standard output as follows –

```
print "Python is really a great language,", "isn't it?"
```

Result:

```
Python is really a great language, isn't it?
```


Reading Keyboard Input

Python provides two built-in functions to read a line of text from standard input, which by default comes from the keyboard. These functions are –

- `raw_input`
- `input`

The *raw_input* Function

The *raw_input*(*[prompt]*) function reads one line from standard input and returns it as a string (removing the trailing newline).

```
str = raw_input("Enter your input: ");  
  
print "Received input is : ", str
```

This prompts you to enter any string and it would display same string on the screen. When I typed "Hello Python!", its output is like this –

```
Enter your input: Hello Python  
  
Received input is : Hello Python
```

The *input* Function

The *input*(*[prompt]*) function is equivalent to *raw_input*, except that it assumes the input is a valid Python expression and returns the evaluated result to you.

```
str = input("Enter your input: ");  
  
print "Received input is : ", str
```

This would produce the following result against the entered input –

```
Enter your input: [x*5 for x in range(2,10,2)]
```

```
Recieved input is : [10, 20, 30, 40]
```

Opening and Closing Files

Until now, you have been reading and writing to the standard input and output. Now, we will see how to use actual data files.

Python provides basic functions and methods necessary to manipulate files by default. You can do most of the file manipulation using a **file** object.

The *open* Function

Before you can read or write a file, you have to open it using Python's built-in *open()* function. This function creates a **file** object, which would be utilized to call other support methods associated with it.

Syntax

```
file object = open(file_name [, access_mode][, buffering])
```

Here are parameter details:

- **file_name:** The `file_name` argument is a string value that contains the name of the file that you want to access.
- **access_mode:** The `access_mode` determines the mode in which the file has to be opened, i.e., read, write, append, etc. A complete list of possible values is given below in the table. This is optional parameter and the default file access mode is read (r).
- **buffering:** If the buffering value is set to 0, no buffering takes place. If the buffering value is 1, line buffering is performed while accessing a file. If you specify the buffering value as an integer greater than 1, then buffering action is performed with the indicated buffer size. If negative, the buffer size is the system default(default behavior).

Here is a list of the different modes of opening a file –

Modes	Description
r	Opens a file for reading only. The file pointer is placed at the beginning of the file. This is the default mode.
rb	Opens a file for reading only in binary format. The file pointer is placed at the beginning of the file. This is the default mode.
r+	Opens a file for both reading and writing. The file pointer placed at the beginning of the file.
rb+	Opens a file for both reading and writing in binary format. The file pointer placed at the beginning of the file.
w	Opens a file for writing only. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
wb	Opens a file for writing only in binary format. Overwrites the file if the file exists. If the file does not exist, creates a new file for writing.
w+	Opens a file for both writing and reading. Overwrites the existing file if the file exists. If the file does not exist, creates a new file for reading and writing.
wb+	Opens a file for both writing and reading in binary format. Overwrites the existing file if the file

	exists. If the file does not exist, creates a new file for reading and writing.
a	Opens a file for appending. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.
ab	Opens a file for appending in binary format. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. If the file does not exist, it creates a new file for writing.
a+	Opens a file for both appending and reading. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.
ab+	Opens a file for both appending and reading in binary format. The file pointer is at the end of the file if the file exists. The file opens in the append mode. If the file does not exist, it creates a new file for reading and writing.

The *file* Object Attributes

Once a file is opened and you have one *file* object, you can get various information related to that file.

Here is a list of all attributes related to file object:

Attribute	Description
file.closed	Returns true if file is closed, false otherwise.

file.mode	Returns access mode with which file was opened.
file.name	Returns name of the file.
file.softspace	Returns false if space explicitly required with print, true otherwise.

Example

```
# Open a file

fo = open("foo.txt", "wb")

print "Name of the file: ", fo.name

print "Closed or not : ", fo.closed

print "Opening mode : ", fo.mode

print "Softspace flag : ", fo.softspace
```

This produces the following result –

```
Name of the file: foo.txt
Closed or not : False
Opening mode : wb
Softspace flag : 0
```

The *close()* Method

The *close()* method of a *file* object flushes any unwritten information and closes the file object, after which no more writing can be done. Python automatically closes a file when the reference object of a file is reassigned to another file. It is a good practice to use the *close()* method to close a file

Syntax

```
fileObject.close();
```

Example

```
# Open a file

fo = open("foo.txt", "wb")

print "Name of the file: ", fo.name

# Close opened file

fo.close()
```

Result –

```
Name of the file: foo.txt
```

Reading and Writing Files

The *file* object provides a set of access methods to make our lives easier. We would see how to use *read()* and *write()* methods to read and write files.

The *write()* Method

The *write()* method writes any string to an open file. It is important to note that Python strings can have binary data and not just text. The *write()* method does not add a newline character ('\n') to the end of the string **Syntax**

```
fileObject.write(string);
```

Here, passed parameter is the content to be written into the opened file. **Example**

```
# Open a file

fo = open("foo.txt", "wb")

fo.write( "Python is a great language.\nYeah its great!!\n");
```

```
# Close opened file
```

```
fo.close()
```

The above method would create *foo.txt* file and would write given content in that file and finally it would close that file. If you would open this file, it would have following content.

```
Python is a great language.
```

```
Yeah its great!!
```

The *read()* Method

The *read()* method reads a string from an open file. It is important to note that Python strings can have binary data. apart from text data.

Syntax

```
fileObject.read([count]);
```

Here, passed parameter is the number of bytes to be read from the opened file. This method starts reading from the beginning of the file and if *count* is missing, then it tries to read as much as possible, maybe until the end of file.

Example

Let's take a file *foo.txt*, which we created above.

```
# Open a file
```

```
fo = open("foo.txt", "r+")
```

```
str = fo.read(10);
```

```
print "Read String is : ", str
```

```
# Close opened file
```

```
fo.close()
```

This produces the following result –

```
Read String is : Python is
```

File Positions

The *tell()* method tells you the current position within the file; in other words, the next read or write will occur at that many bytes from the beginning of the file.

32

The *seek(offset[, from])* method changes the current file position. The *offset* argument indicates the number of bytes to be moved. The *from* argument specifies the reference position from where the bytes are to be moved.

If *from* is set to 0, it means use the beginning of the file as the reference position and 1 means use the current position as the reference position and if it is set to 2 then the end of the file would be taken as the reference position.

Example

Let us take a file *foo.txt*, which we created above.

```
# Open a file

fo = open("foo.txt", "r+")

str = fo.read(10);

print "Read String is : ", str


# Check current position

position = fo.tell();
```



```

print "Current file position : ", position

# Reposition pointer at the beginning once again

position = fo.seek(0, 0);

str = fo.read(10);

print "Again read String is : ", str

# Close opened file

fo.close()

```

This produces the following result –

```

Read String is : Python is

Current file position : 10

Again read String is : Python is

```

Renaming and Deleting Files

Python **os** module provides methods that help you perform file-processing operations, such as renaming and deleting files.

To use this module you need to import it first and then you can call any related functions.

The `rename()` Method

The *rename()* method takes two arguments, the current filename and the new filename.

Syntax

```

os.rename(current_file_name, new_file_name)

```

Example

Following is the example to rename an existing file *test1.txt*:

```
import os

# Rename a file from test1.txt to test2.txt

os.rename( "test1.txt", "test2.txt" )
```

The *remove()* Method

You can use the *remove()* method to delete files by supplying the name of the file to be deleted as the argument.

Syntax

```
os.remove(file_name)
```

Example

Following is the example to delete an existing file *test2.txt* –

```
#!/usr/bin/python

import os

# Delete file test2.txt

os.remove("text2.txt")
```

Directories in Python

All files are contained within various directories, and Python has no problem handling these too. The **os** module has several methods that help you create, remove, and change directories.

The *mkdir()* Method

You can use the *mkdir()* method of the **os** module to create directories in the current directory. You need to supply an argument to this method which contains the name of the directory to be created.

Syntax

```
os.mkdir("newdir")
```

Example

Following is the example to create a directory *test* in the current directory –

```
#!/usr/bin/python
import os

# Create a directory "test"

os.mkdir("test")
```

The *chdir()* Method

You can use the *chdir()* method to change the current directory. The *chdir()* method takes an argument, which is the name of the directory that you want to make the current directory.

Syntax

```
os.chdir("newdir")
```

Example

Following is the example to go into *"/home/newdir"* directory –

```
#!/usr/bin/python

import os
```

```
# Changing a directory to "/home/newdir"
```

```
os.chdir("/home/newdir")
```

The *getcwd()* Method

The *getcwd()* method displays the current working directory.

Syntax

```
os.getcwd()
```

Example

Following is the example to give current directory –

```
import os
```

```
# This would give location of the current directory
```

```
os.getcwd()
```

The *rmdir()* Method

The *rmdir()* method deletes the directory, which is passed as an argument in the method.

Before removing a directory, all the contents in it should be removed.

Syntax:

```
os.rmdir('dirname')
```

Example

Following is the example to remove `"/tmp/test"` directory. It is required to give fully qualified name of the directory, otherwise it would search for that directory in the current directory.

```
import os

# This would remove "/tmp/test" directory.

os.rmdir( "/tmp/test" )
```

File & Directory Related Methods

There are three important sources, which provide a wide range of utility methods to handle and manipulate files & directories on Windows and Unix operating systems. They are as follows –

- File Object Methods: The *file* object provides functions to manipulate files.
- OS Object Methods: This provides methods to process files as well as directories.

Python provides two very important features to handle any unexpected error in your Python programs and to add debugging capabilities in them –

- **Exception Handling**: This would be covered in this tutorial. Here is a list standard Exceptions available in Python: Standard Exceptions.
- **Assertions**: This would be covered in Assertions in Python

List of Standard Exceptions –

EXCEPTION NAME	DESCRIPTION
Exception	Base class for all exceptions
StopIteration	Raised when the next() method of an iterator does not point to any object.

SystemExit	Raised by the sys.exit() function.
StandardError	Base class for all built-in exceptions except StopIteration and SystemExit.
ArithmeticError	Base class for all errors that occur for numeric calculation.
OverflowError	Raised when a calculation exceeds maximum limit for a numeric type.
FloatingPointError	Raised when a floating point calculation fails.
ZeroDivisionError	Raised when division or modulo by zero takes place for all numeric types.
AssertionError	Raised in case of failure of the Assert statement.
AttributeError	Raised in case of failure of attribute reference or assignment.
EOFError	Raised when there is no input from either the raw_input() or input() function and the end of file is reached.
ImportError	Raised when an import statement fails.
KeyboardInterrupt	Raised when the user interrupts program execution, usually by pressing Ctrl+c.

LookupError	Base class for all lookup errors.
IndexError	Raised when an index is not found in a sequence.
KeyError	Raised when the specified key is not found in the dictionary.
NameError	Raised when an identifier is not found in the local or global namespace.
UnboundLocalError	Raised when trying to access a local variable in a function or method but no value has been assigned to it.
EnvironmentError	Base class for all exceptions that occur outside the Python environment.
IOError	Raised when an input/ output operation fails, such as the print statement or the open() function when trying to open a file that does not exist.
IOError	Raised for operating system-related errors.
SyntaxError	Raised when there is an error in Python syntax.
IndentationError	Raised when indentation is not specified properly.
SystemError	Raised when the interpreter finds an internal problem, but when this error is encountered the Python interpreter does not exit.
SystemExit	Raised when Python interpreter is quit by using the sys.exit() function. If not handled in the code, causes the interpreter to exit.
TypeError	Raised when an operation or function is attempted that is invalid for the specified data type.

ValueError	Raised when the built-in function for a data type has the valid type of arguments, but the arguments have invalid values specified.
RuntimeError	Raised when a generated error does not fall into any category.
NotImplementedError	Raised when an abstract method that needs to be implemented in an inherited class is not actually implemented.

What is Exception?

An exception is an event, which occurs during the execution of a program that disrupts the normal flow of the program's instructions. In general, when a Python script encounters a situation that it cannot cope with, it raises an exception. An exception is a Python object that represents an error.

When a Python script raises an exception, it must either handle the exception immediately otherwise it terminates and quits.

Handling an exception

If you have some *suspicious* code that may raise an exception, you can defend your program by placing the suspicious code in a **try:** block. After the try: block, include an **except:** statement, followed by a block of code which handles the problem as elegantly as possible.

The Python standard for database interfaces is the Python DB-API. Most Python database interfaces adhere to this standard.

You can choose the right database for your application. Python Database API supports a wide range of database servers such as –

- GadFly
- mSQL
- MySQL

- PostgreSQL
- Microsoft SQL Server 2000
- Informix
- Interbase
- Oracle
- Systembase

The DB API provides a minimal standard for working with databases using Python structures and syntax wherever possible. This API includes the following:

- Importing the API module.
- Acquiring a connection with the database.
- Issuing SQL statements and stored procedures.
- Closing the connection

9.2 SOURCE CODE

HOME-PAGE

```
#!/usr/bin/env python

"""Django's command-line utility for administrative tasks."""

import os

import sys

def main():

    """Run administrative tasks."""

    os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'deepside.settings')

    try:
```

```

from django.core.management import execute_from_command_line

except ImportError as exc:

    raise ImportError(

        "Couldn't import Django. Are you sure it's installed and "

        "available on your PYTHONPATH environment variable? Did you "

        "forget to activate a virtual environment?"

    ) from exc

execute_from_command_line(sys.argv)

```

```

if __name__ == '__main__':

    main[16:06, 17/05/2024] +91 6304 261 698: {% extends 'RUser/design.html' %}

    {% block userblock %}

```

```

<link rel="icon" href="images/icon.png" type="image/x-icon" />

```

```

    <link href="https://fonts.googleapis.com/css?family=Lobster" rel="stylesheet">

    <link href="https://fonts.googleapis.com/css?family=Righteous"
rel="stylesheet">

    <link href="https://fonts.googleapis.com/css?family=Fredoka+One" rel="stylesheet">

```

```

<style>

    body {background-color:#000000;}

    .container-fluid {padding:50px;}

    .container{background-color:white;padding:50px; }

    #title{font-family: 'Fredoka One', cursive;

}

    .text-uppercase{

    font-family: 'Righteous', cursive;

    }

    .tweettext{

border: 2px solid yellowgreen;

width: 904px;

height: 202px;

overflow: scroll;

background-color;;

}

    .style1 {

...

[16:06,      17/05/2024]      +91      6304      261      698:      <link
href="//maxcdn.bootstrapcdn.com/bootstrap/4.0.0/css/bootstrap.min.css"      rel="stylesheet"
id="bootstrap-css">

{% load static %}

```

ENABLE USER:

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<title>Login</title>
```

```
<meta charset="utf-8">
```

```
<meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">
```

```
<head>
```

```
<link rel="icon" href="images/icon.png" type="image/x-icon" />
```

```
<link href="https://fonts.googleapis.com/css?family=Lobster" rel="stylesheet">
```

```
<link href="https://fonts.googleapis.com/css?family=Righteous" rel="stylesheet">
```

```
<link href="https://fonts.googleapis.com/css?family=Fredoka+One" rel="stylesheet">
```

```
<style>
```

```
body {background: url("{% static 'bg.jpg' %}");
```

```
background-color:#eee;}
```

```
.container-fluid {padding:50px;}
```

```
.container{bac...
```

```
[16:06, 17/05/2024] +91 6304 261 698: {% extends 'SProvider/design1.html' %}
```

```
{% block researchblock %}
```

```
<link rel="icon" href="images/icon.png" type="image/x-icon" />
```

```
<link href="https://fonts.googleapis.com/css?family=Lobster" rel="stylesheet">
```

```
<link href="https://fonts.googleapis.com/css?family=Righteous" rel="stylesheet">
```

```
<link href="https://fonts.googleapis.com/css?family=Fredoka+One" rel="stylesheet">
```

```
<style>
```

```
body {background-color:#000000;}
```

```
.container-fluid {padding:50px;}
```

```
.container{background-color:white;padding:50px; }
```

```
#title{font-family: 'Fredoka One', cursive;
```

```
}
```

```
.text-uppercase{
```

```
font-family: 'Righteous', cursive;
```

```
}
```

```
input{
```

```

        font-family:'Russo One', sans-serif;

        font-size:15px;

    }

    .style1 {color: #FF0000}

</style>

<body>

<div class="container-fluid">

    <div class="container">

        <div class="row">

            <div class="col-md-5">

                <form role="form" method="POST" >

                    {% csrf_token %}

                    <fieldset>

                        <p      class="text-uppercase      pull-center
style1">VIEW ALL REMOTE USERS !!! </p>

                        <div class="viewclients">

                            <table>

                                <tr>

                                    <th      bgcolor="#FF0000"
style="color:yellow">USER NAME</th>

                                    <th      bgcolor="#FF0000"
style="color:yellow">EMAIL</th>

```

```

                                <th      bgcolor="#FF0000"
style="color:yellow">Gender</th>

                                <th      bgcolor="#FF0000"
style="color:yellow">Address</th>

                                <th      bgcolor="#FF0000"
style="color:yellow">Mob No</th>

                                <th      bgcolor="#FF0000"
style="color:yellow">Country</th>

                                <th      bgcolor="#FF0000"
style="color:yellow">State</th>

                                <th      bgcolor="#FF0000"
style="color:yellow">City</th>

                                </tr>

                                <tr>

                                {% for o in objects %}

                                <td      bgcolor="#FFFFFF"
style="color:white"><span class="style1">{{o.username}}</span></td>

                                <td      bgcolor="#FFFFFF"
style="color:white"><span class="style1">{{o.email}}</span></td>

                                <td      bgcolor="#FFFFFF"
style="color:white"><span class="style1">{{o.gender}}</span></td>

                                <td      bgcolor="#FFFFFF"
style="color:white"><span class="style1">{{o.address}}</span></td>

                                <td      bgcolor="#FFFFFF"
style="color:white"><span class="style1">{{o.phoneno}}</span></td>

                                <td      bgcolor="#FFFFFF"
style="color:white"><span class="style1">{{o.country}}</span></td>

```

```
 <span class="style1">{{o.state}}</span></td>   <span class="style1">{{o.city}}</span></td>  </tr>  {% endfor %}  </table>  </div>  </fieldset>  </form>  </div>  <div class="col-md-2">  <!--null-->  </div>  </div>  </div>  </div>  {% endblock %} | |
```


CHAPTER-10

RESULTS/DISCUSSION

10.1 SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub-assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

TYPES OF TESTS

UNIT TESTING

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application. It is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

INTEGRATION TESTING

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfactory, as shown by successful unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

FUNCTIONAL TEST

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

- Valid Input : identified classes of valid input must be accepted.
- Invalid Input : identified classes of invalid input must be rejected.
- Functions : identified functions must be exercised.
- Output : identified classes of application outputs must be exercised.
- Systems/Procedures : interfacing systems or procedures must be invoked.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

SYSTEM TEST

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration-oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

WHITE BOX TESTING

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

BLACK BOX TESTING

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

UNIT TESTING

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

TEST STRATEGY AND APPROACH

Field testing will be performed manually and functional tests will be written in detail.

Test objectives

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

Features to be tested

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

ACCEPTANCE TESTING

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

10.1.1 TEST CASES:

Test case1:

Test case for Login form:

FUNCTION:	LOGIN
EXPECTED RESULTS:	Should Validate the user and check his existence in database
ACTUAL RESULTS:	Validate the user and checking the user against the database
LOW PRIORITY	No
HIGH PRIORITY	Yes

Test case2:

Test case for User Registration form:

FUNCTION	External client user
----------	----------------------

EXPECTED RESULTS:	Should check if all the fields are filled by the user and saving the user to database.
ACTUAL RESULTS:	Checking whether all the fields are filled by user or not through validations and saving user.
LOW PRIORITY	No
HIGH PRIORITY	Yes

Test case3:

Test case for Change Password:

When the old password does not match with the new password , then this results in displaying an error message as “ OLD PASSWORD DOES NOT MATCH WITH THE NEW PASSWORD”.

Test case 4:

Test case for Forget Password:

When a user forgets his password he is asked to enter Login name, ZIP code, Mobile number. If these are matched with the already stored ones then user will get his Original password.

Module	Functionality	Test Case	Expected Results	Actual Results	Result	Priority
--------	---------------	-----------	------------------	----------------	--------	----------

User	Login Usecase	1. Navigate To Www.Sample.Co m 2. Click On Submit Button Without Entering Username and Password	A Validation Should Be As Below “Please Enter Valid Username & Password”	A Validation Has Been Populated As Expected	Pass	High
		1. aNavigate To Www.Sample.Co m 2. Click On Submit Button With Out Filling Password And With Valid Username	A Validation Should Be As Below “Please Enter Valid Password Or Password Field Can Not Be Empty “	A Validation Is Shown As Expected	Pass	High
		1. NNavigate To Www.Sample.Co m 2. Enter Both Username And Password Wrong And Hit Enter	A Validation Shown As Below “The Username Entered Is Wrong”	A Validation Is Shown As Expected	Pass	High

		1. Navigate To Www.Sample.Co m	Validate Username And Password In DataBase And Once Has Been	Main Page/Pass Home Page		High
		2. Enter Validate Username And Password And Click On Submit	If They Correct Then Show The Main Page	Displayed		

10.2 SCREENSHOTS



FIG-1-Homepage



FIG-2-Login :enable user

REGISTER NOW!

REGISTER YOUR DETAILS HERE !!!

Enter Username	User Name	Enter Password	Password
Enter EMail Id	Enter Email	Enter Address	Enter Address
Enter Gender	---Select Gender ---	Enter Mobile Number	Enter Mobile Number
Enter Country Name	Enter Country Name	Enter State Name	Enter State Name
Enter City Name	Enter City Name	Register	

Registered Status ::

FIG-3-Register: external client user

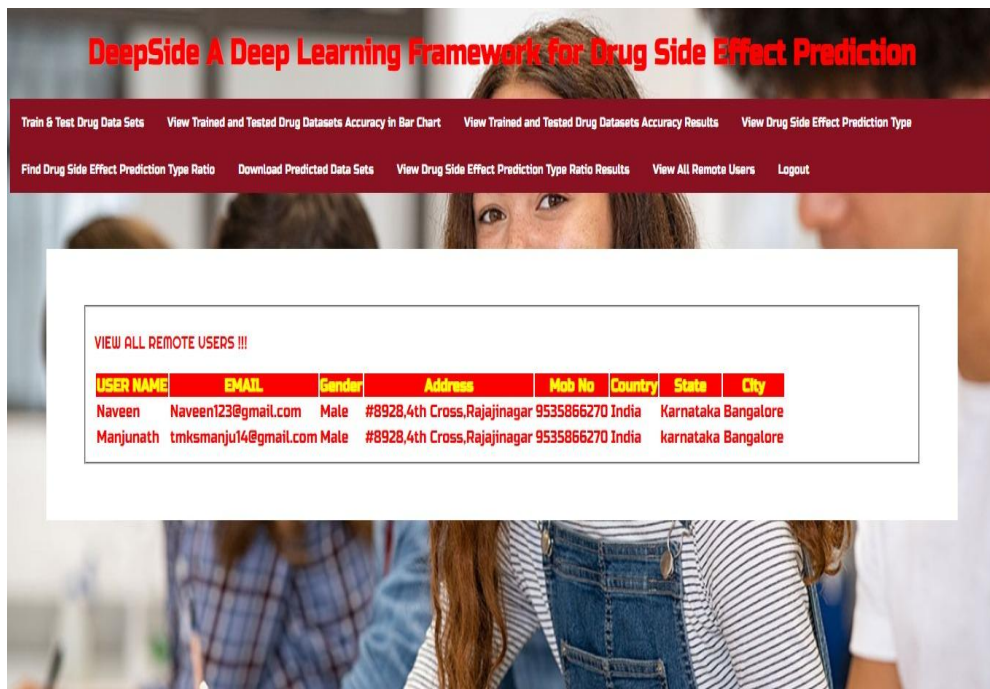


FIG-4:test-train drug data sets

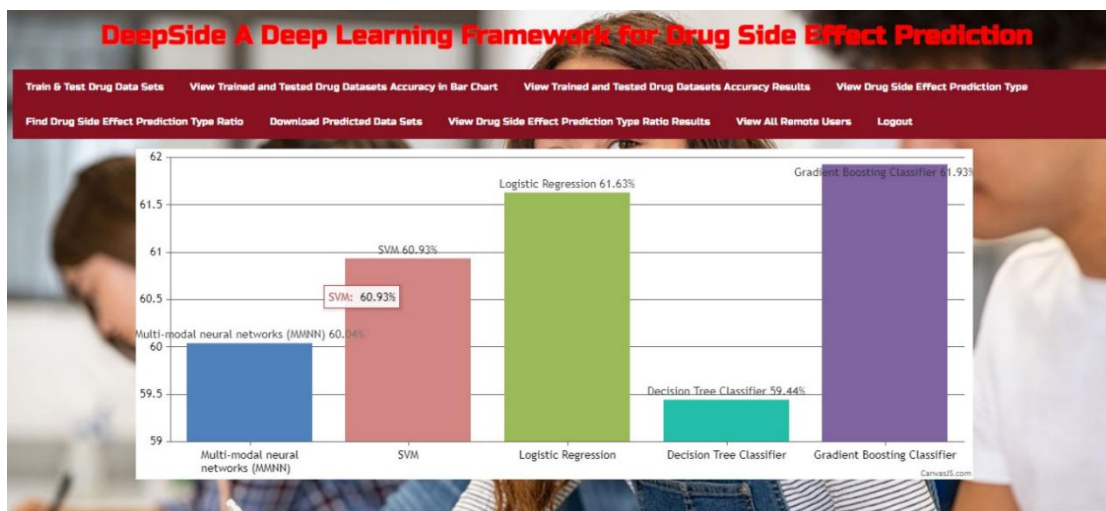


FIG-5: View trained and tested drug datasets accuracy in bar chart

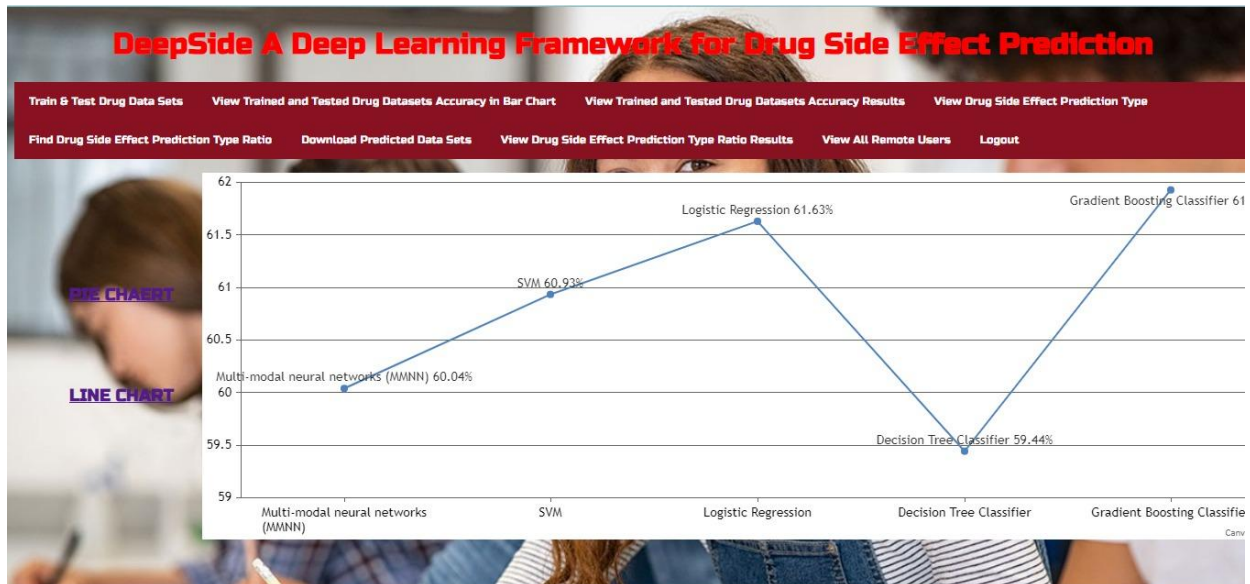


FIG-6:View trained and tested drug datasets accuracy results

DeepSide A Deep Learning Framework for Drug Side Effect Prediction

Train & Test Drug Data Sets | View Trained and Tested Drug Datasets Accuracy in Bar Chart | View Trained and Tested Drug Datasets Accuracy Results | View Drug Side Effect Prediction Type

Find Drug Side Effect Prediction Type Ratio | Download Predicted Data Sets | View Drug Side Effect Prediction Type Ratio Results | View All Remote Users | Logout

View Drug Side Effect Prediction Type

uid	Drug_Name	Condition	Prediction
8ilzm51q	Aripiprazole	Bipolar Disorde	High Side Effect Found
wafvnbdu	Keppra	Epilepsy	Low Side Effect Found
lhm6erzy	Pentasa	Crohn's Disease	Low Side Effect Found
08mep8hm	Trimethoprim	Urinary Tract Infection	High Side Effect Found
tiyjmh7	Tioconazole	Vaginal Yeast Infection	Low Side Effect Found
y9sel25v	Rizatriptan	Migraine	High Side Effect Found

FIG-7 View drug side effect prediction type

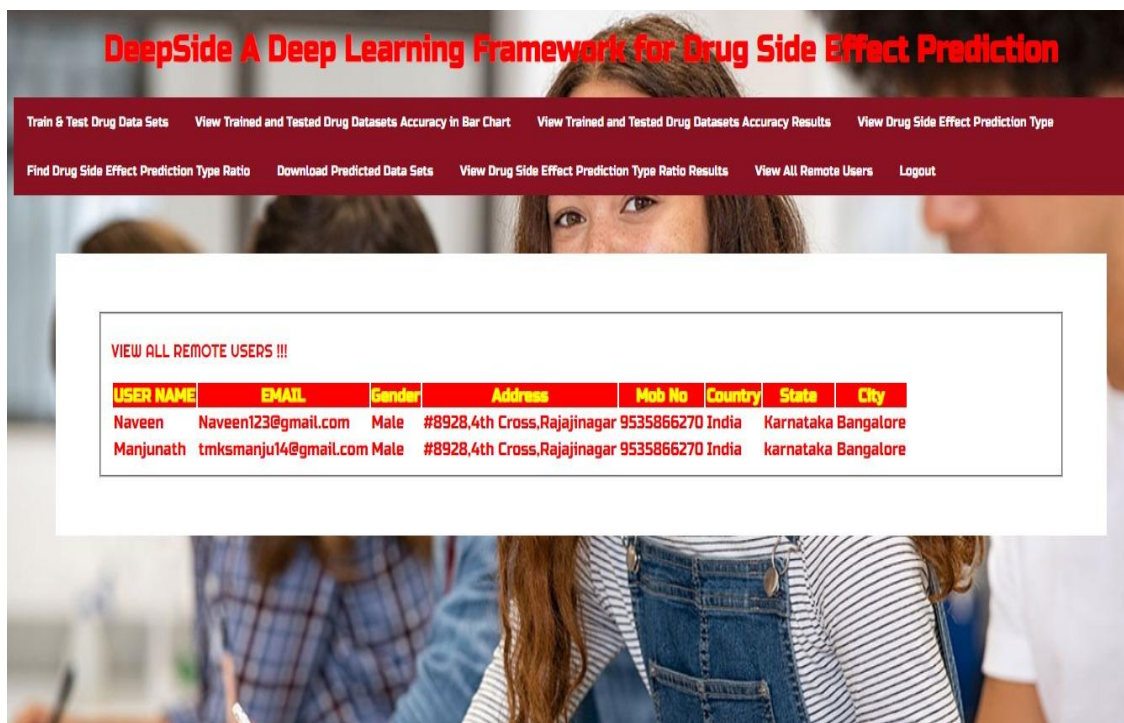


FIG-8 View all remote users

CHAPTER-11

CONCLUSION

11.1 CONCLUSION

In this paper, since the phenotypic numerical features contained in agricultural texts have practical significance, combined with the experimental tasks of this study, we proposed a classification method of agricultural texts based on dynamic fusion of multiple features. Firstly, three convolution kernels with different sizes are used to obtain more abundant local semantic features at different levels of wheat cold resistance text, and Bi-LSTM with attention-fusion mechanism is used to obtain important semantic expressions between contexts in the global scope. Then independent numerical features are constructed by using the text of wheat traits with numerical expression. Attention mechanism is introduced again in the process of multi-feature dynamic fusion to dynamically adjust the weights of different semantic expressions in the process of feature fusion and capture the key information beneficial to text classification in different levels of features. The performance of agricultural text classification shows that the proposed multi-feature dynamic fusion method fully excavates the potential semantic features in the text data, which is effective for the agricultural text classification task with phenotypic values. The next work will expand the data set and sample equalization to improve the performance of wheat cold resistance text classification and recognition.

11.2 FUTURE SCOPE

The traditional text classification methods, which treats the values in agricultural text as characters, will lose the original semantic expression of numerical features. In order to fully mine the deep latent semantic features in agricultural text, a novel text classification method based on multivariate feature dynamic fusion is proposed. The Bi-directional Long Short Term Memory network (Bi-LSTM) model with attention mechanism was used to extract the global key semantic features of the text; the multiple windows Convolution Neural Network were used to extract the local semantic information of the text at different levels; Numerical value features containing essential semantic expression were extracted by artificial method to construct the

numerical value feature vector. By introducing the attention mechanism to dynamically fuse the extracted multiple semantic features, which can further enrich the deep semantic expression of agricultural text and effectively improve the effect of agricultural text classification with phenotypic numerical type.

CHAPTER-12

REFERENCES

- [1. Atias, N., Sharan, R.: An algorithmic framework for predicting side effects of drugs. *Journal of Computational Biology* 18(3), 207{218 (2011)
2. Bresso, E., Grisoni, R., Marchetti, G., Karaboga, A.S., Souchet, M., Devignes, M.D., Smail-Tabbone, M.: Integrative relational machine-learning for understanding drug side-effect profiles. *BMC bioinformatics* 14(1), 207 (2013)
3. Cai, M.C., Xu, Q., Pan, Y.J., Pan, W., Ji, N., Li, Y.B., Jin, H.J., Liu, K., Ji, Z.L.: Adres: an ontology database for aiding standardization and hierarchical classification of adverse drug reaction terms. *Nucleic acids research* 43(D1), D907{D913 (2014)
4. Dey, S., Luo, H., Fokoue, A., Hu, J., Zhang, P.: Predicting adverse drug reactions through interpretable deep learning framework. *BMC Bioinformatics* 19 (12 2018). <https://doi.org/10.1186/s12859-018-2544-0>
5. Dimitri, G.M., Lio, P.: Drugclust: A machine learning approach for drugs side effects prediction. *Computational Biology and Chemistry* 68, 204 { 210 (2017). <https://doi.org/https://doi.org/10.1016/j.compbiolchem.2017.03.008>
6. Groopman, J.E., Itri, L.M.: Chemotherapy-induced anemia in adults: incidence and treatment. *Journal of the National Cancer Institute* 91(19), 1616{1634 (1999)
7. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (June 2016)
8. Huang, L.C., Wu, X., Chen, J.Y.: Predicting adverse side effects of drugs. *BMC genomics* 12(5), S11 (2011)
9. Huang, L.C., Wu, X., Chen, J.Y.: Predicting adverse drug reaction profiles by integrating protein interaction networks with drug structures. *Proteomics* 13(2), 313{324 (2013)
10. Ioffe, S., Szegedy, C.: Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint arXiv:1502.03167* (2015)
11. Kim, S., Thiessen, P.A., Bolton, E.E., Chen, J., Fu, G., Gindulyte, A., Han, L., He, J., He, S., Shoemaker, B.A., et al.: Pubchem substance and compound databases. *Nucleic acids research* 44(D1), D1202{D1213 (2015)

12. Krizhevsky, A., Sutskever, I., Hinton, G.E.: Imagenet classification with deep convolutional neural networks. In: Advances in neural information processing systems. pp. 1097{1105 (2012)
13. Kuhn, M., Letunic, I., Jensen, L.J., Bork, P.: The sider database of drugs and side effects. Nucleic acids research 44(D1), D1075{D1079 (2015)
14. Landrum, G., et al.: Rdkit: Open-source cheminformatics (2006)
15. Lee, W., Huang, J., Chang, H., Lee, K., Lai, C.: Predicting drug side effects using data analytics and the integration of multiple data sources. IEEE Access 5, 20449{20462 (2017). <https://doi.org/10.1109/ACCESS.2017.2755045>
16. Li, Y., Yuan, Y.: Convergence analysis of two-layer neural networks with relu activation. In: Advances in Neural Information Processing Systems. pp. 597{607 (2017)
17. Liu, M., Wu, Y., Chen, Y., Sun, J., Zhao, Z., Chen, X.w., Matheny, M.E., Xu, H.: Large-scale prediction of adverse drug reactions using chemical, biological, and phenotypic properties of drugs. Journal of the American Medical Informatics Association 19(e1), e28{e35 (2012)
18. Lopes, C.E., Langoski, G., Klein, T., Ferrari, P.C., Farago, P.V.: A simple hplc method for the determination of halcinonide in lipid nanoparticles: development, validation, encapsulation efficiency, and in vitro drug permeation. Brazilian Journal of Pharmaceutical Sciences 53(2) (2017)
19. Mizutani, S., Pauwels, E., Stoven, V., Goto, S., Yamanishi, Y.: Relating drug{protein interaction network with drug side effects. Bioinformatics 28(18), i522{i528 (2012)
20. O'Boyle, N.M., Banck, M., James, C.A., Morley, C., Vandermeersch, T., Hutchison, G.R.: Open babel: An open chemical toolbox. Journal of cheminformatics 3(1), 33 (2011)
21. Okubo, S., Nakatani, K., Nishiya, K.: Gastrointestinal symptoms associated with enteric-coated sulfasalazine (azul_dine en tablets). Modern rheumatology 12(3), 0226{0229 (2002)
22. Pal, S.K., Mitra, S.: Multilayer perceptron, fuzzy sets, and classification. IEEE Transactions on Neural Networks 3(5), 683{697 (Sep 1992). <https://doi.org/10.1109/72.159058>
23. Pauwels, E., Stoven, V., Yamanishi, Y.: Predicting drug side-effect profiles: a chemical fragment-based approach. BMC bioinformatics 12(1), 169 (2011)
24. Polatti, F., Viazzo, F., Colleoni, R., Nappi, R.E.: Uterine myoma in postmenopause: a comparison between two therapeutic schedules of hrt. Maturitas 37(1), 27{32 (2000)

25. Scheiber, J., Jenkins, J.L., Sukuru, S.C.K., Bender, A., Mikhailov, D., Milik, M., Azzaoui, K., Whitebread, S., Hamon, J., Urban, L., et al.: Mapping adverse drug reactions in chemical space. *Journal of medicinal chemistry* 52(9), 3103–3107 (2009)